基于 Buildroot 系统

Qt 开发环境搭建

正点原子

正点原子公司名称 ：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛 ：http://www.openedv.com/forum.php

正点原子淘宝店铺 ：https://openedv.taobao.com

正点原子官方网站 ：www.alientek.com

正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请关注正点原子公众号，资料发布更新我们会通知。

扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿：基于【正点原子】ATK-DLRK3568_Qt 开发环境搭建 V1.1	正点原子 Linux 团队	正点原子 Linux 团队	2024.03.23
V1.1	适配 ATK-DLRK3506	正点原子 Linux 团队	正点原子 Linux 团队	2025.04.10

目录

文档更新说明 3
免责声明 5
前言 6
第一章 Ubuntu 下安装 Qt Creator 7
1.1 安装 Qt 8
1.2 配置 Qt Creator 输入中文 16
1.2.1 配置 Ubuntu 中文环境 16
1.2.2 配置中文输入法 19
1.3 Qt Creator 简单使用 21
1.3.1 Qt Creator 界面组成 21
1.3.2 Qt Creator 设置 22
1.4 第一个 Qt 程序 24
1.4.1 新建一个项目 24
1.4.2 项目文件介绍 29
1.4.3 修改 ui 文件显示 hello world 36
1.4.4 项目编译&调试&运行 37
第二章 交叉编译环境搭建 40
2.1 安装交叉编译工具链 41
2.2 命令行交叉编译 Qt 项目 41
2.2.1 查看 qmake 与 Qt 版本 41
2.2.2 编译 Qt 程序 41
2.3 QtCreator 开发嵌入式平台 Qt 应用程序 42
2.3.1 配置 ARM 交叉编译器 42
2.3.2 配置 qmake 45
2.3.3 配置 Kit 46
2.3.4 编译测试 47
2.4 远程部署 Qt 应用程序 48
2.4.1 配置运行设备 49
2.4.2 运行设备测试 56
附录 A 常见问题 58
部署失败问题 58

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

正点原子资料下载中心：http://www.openedv.com/docs/index.html

前言

本文旨在正点原子用户快速搭建 Qt 开发环境。基本适用于正点原子所有 Linux 开发板，只是交叉编译器的不同，所以请用户根据对应的开发板，找到对应的编译器，参考这份文档完成各自的嵌入式 Qt 环境搭建。

本用户文档所用的环境如下：

- 正点原子提供的交叉编译器：以 ATK-DLRK3568 为例（正点原子的嵌入式 Qt 环境搭建都差不多，可参考），请大家到对应的资料目录找到“.run”后缀的安装包。

开发板网盘 A-基础资料->05、开发工具->03、交叉编译工具 atk-dlrk3568-toolchain-arm-buildroot-linux-gnueabihf-x86_64.run

- Ubuntu20.04 版本，（请勿在其他 Ubuntu（如 ubuntu22）上验证，若出现不可预知的错误，请自行解决。如果你非常熟悉，则可以在 Ubuntu20 以上版本上随意安装。）：

开发板网盘 A-基础资料->04、软件->ubuntu-20.04.4-desktop-amd64.iso

读者需要熟悉使用 Ubuntu。本教程需要在 Ubuntu 安装的软件汇总。教程里在 Ubuntu 实验需要安装的工具汇总，请大家提前安装。否则在 Qt Creator 编译程序时可能会报错，一般都是你没有安装下面的某些工具！

```
sudo apt-get install gcc g++ lsb-core lib32stdc++6 libglu1-mesa-dev \
gstreamer1.0-plugins-base gstreamer1.0-plugins-bad gstreamer1.0-libav \
gstreamer1.0-plugins-good gstreamer1.0-plugins-ugly gstreamer1.0-pulseaudio \
cmake openssh-server net-tools
```

第一章 Ubuntu 下安装 Qt Creator

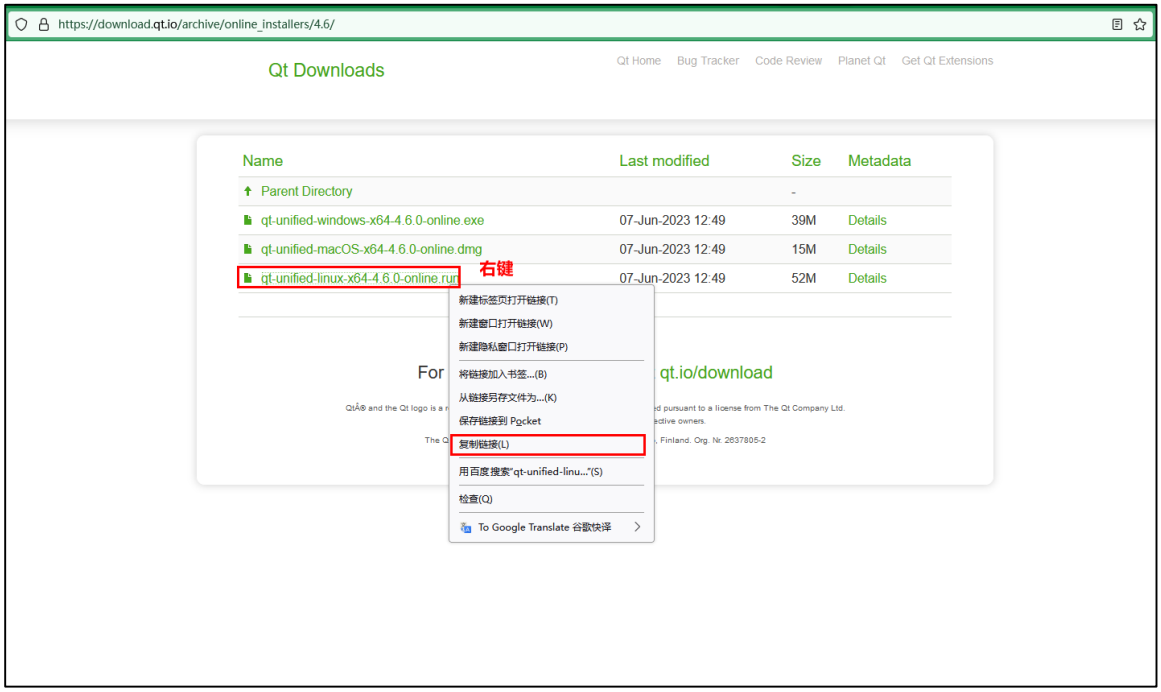
虽说 Qt 是跨平台的，如果你想开发嵌入式 Qt，那么你还得在 Ubuntu 下安装 Qt，因为 Linux 交叉编译器只能在 Linux 才能用的起来！所以我们一起在 Ubuntu 下安装 Qt 吧！

1.1 安装 Qt

请注意，由于 Qt 官方网站受限，1.1 小节安装 Qt 的方法不能再使用了，可以跳过，在线安装已经无法下载 Qt，所以请安装 Qt 的最后一个离线包 Qt5.12.12，请查阅网盘资料中 10、用户手册/辅助文档《【正点原子】Ubuntu 安装 Qt 安装参考手册》安装离线 Qt。

进入 https://download.qt.io/archive/online_installers/4.6/下载页面（注意如果找不到下载链接，我们就进行 http://download.qt.io 这个顶层目录一个个目录找，因为 Qt 下载链接会变动），选择 Linux 的安装包下载。使用迅雷下载再拷贝过去 Ubuntu 虚拟机或者直接复制链接地址到 Ubuntu 虚拟机下载。

说明：从 Qt5.15 官方没有独自提供 Linux 版本/Window 的 Qt 给我们安装，所我们需要在线下载 Qt 来安装，写此文档时，在线安装软件版本为 4.6.0 为最新。若日后 Qt 会更新在线软件版本请根据个人判断是否需要使用最新版本来安装，笔者测试的是 4.6.0 版本的 Linux X64 版本的在线安装 Qt。切记此版本需要使用 Ubuntu20.04 也就是需要 Ubuntu20 以上来安装，Ubuntu18 是不适用的（因为笔者测试过）。



如下图，复制链接下载地址到 Ubuntu 虚拟机终端下使用指令 wget 下载，https://download.qt.io/archive/online_installers/4.6/qt-unified-linux-x64-4.6.0-online.run。

```
wget https://download.qt.io/archive/online_installers/4.6/qt-unified-linux-x64-4.6.0-online.run
```



赋予可执行权限，加上 sudo 权限进入安装，输入密码，这样会安装在/opt 目录下。

```
chmod +x qt-unified-linux-x64-4.6.0-online.run
```

安装 Qt 需要登录你的 Qt 帐号。如果你没有 Qt 帐号，那么你还需要需要到 Qt 官方网站 https://www.qt.io/去注册一个 Qt 帐号。此时你的 Ubuntu 需要联网。

首先，我们需要安装 libxcb-xinerama0，Qt 在线安装需要安装下面的库

```
sudo apt-get install libxcb-xinerama0
```

在线安装 Qt，加入参数--mirror https://mirrors.aliyun.com/qt 这样指定镜像源，不多说，我们直接指定阿里云的镜像源，若阿里云源不能用时，这些源请一个个测试。

备用镜像源地址，若镜像源不可用时我们还有备用的镜像源地址，防止下载出错，或者服务器开小差。

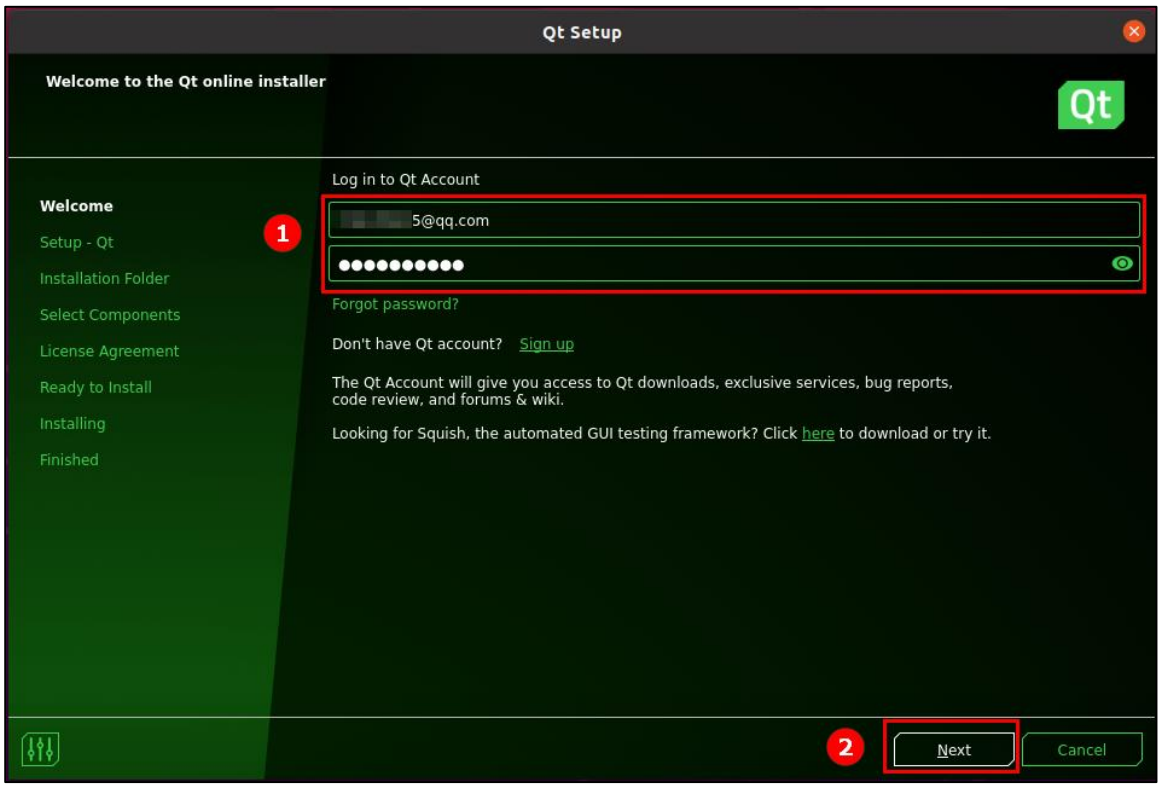
```
https://mirror.nju.edu.cn/qt                // 南京大学源
http://mirrors.ustc.edu.cn/qtproject        // 中科大源
https://mirrors.cnnic.cn/qt                 // 开源软件源
https://download.qt.io                      // Qt 官方源
```

建议与笔者一样，使用 sudo 权限来安装，默认会安装在/opt/Qt 这个路径。

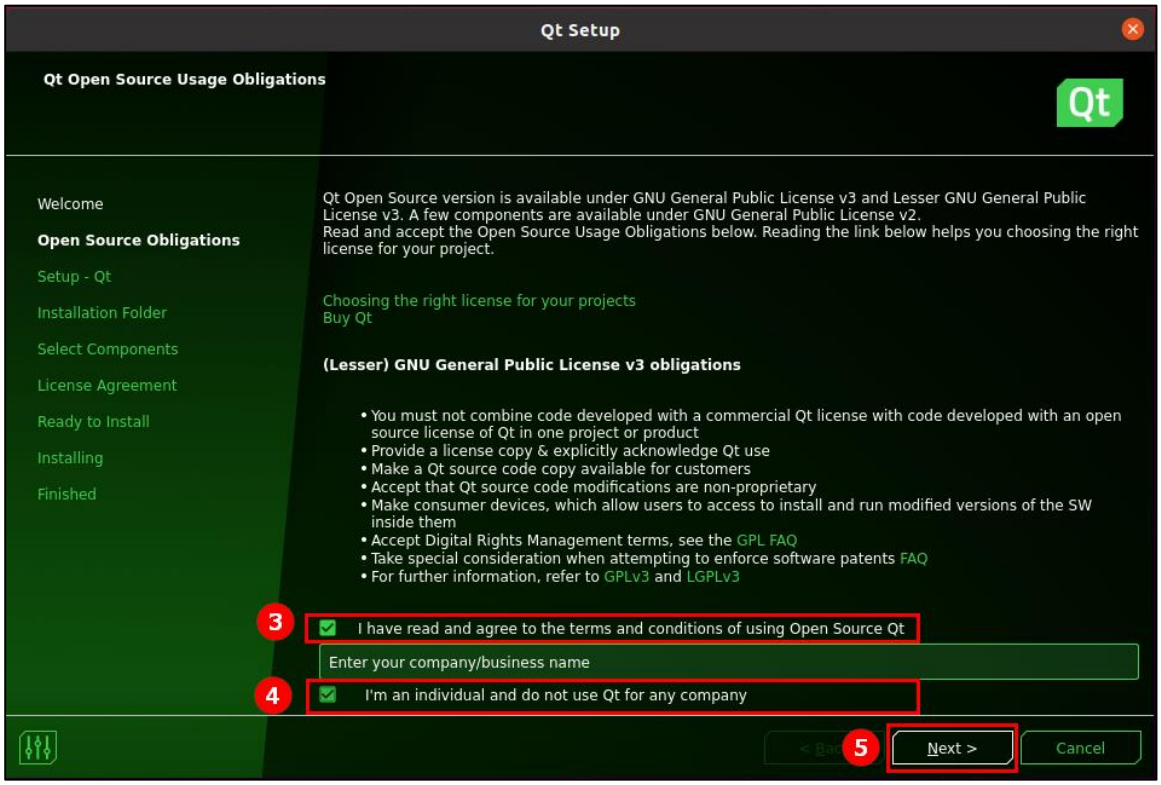
```
sudo ./qt-unified-linux-x64-4.6.0-online.run --mirror https://mirrors.aliyun.com/qt
```

```
alientek@ubuntu:~$ chmod +x qt-unified-linux-x64-4.6.0-online.run
alientek@ubuntu:~$ sudo ./qt-unified-linux-x64-4.6.0-online.run --mirror https://mirrors.aliyun.com/qt
[sudo] alientek 的密码:
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-root'
```

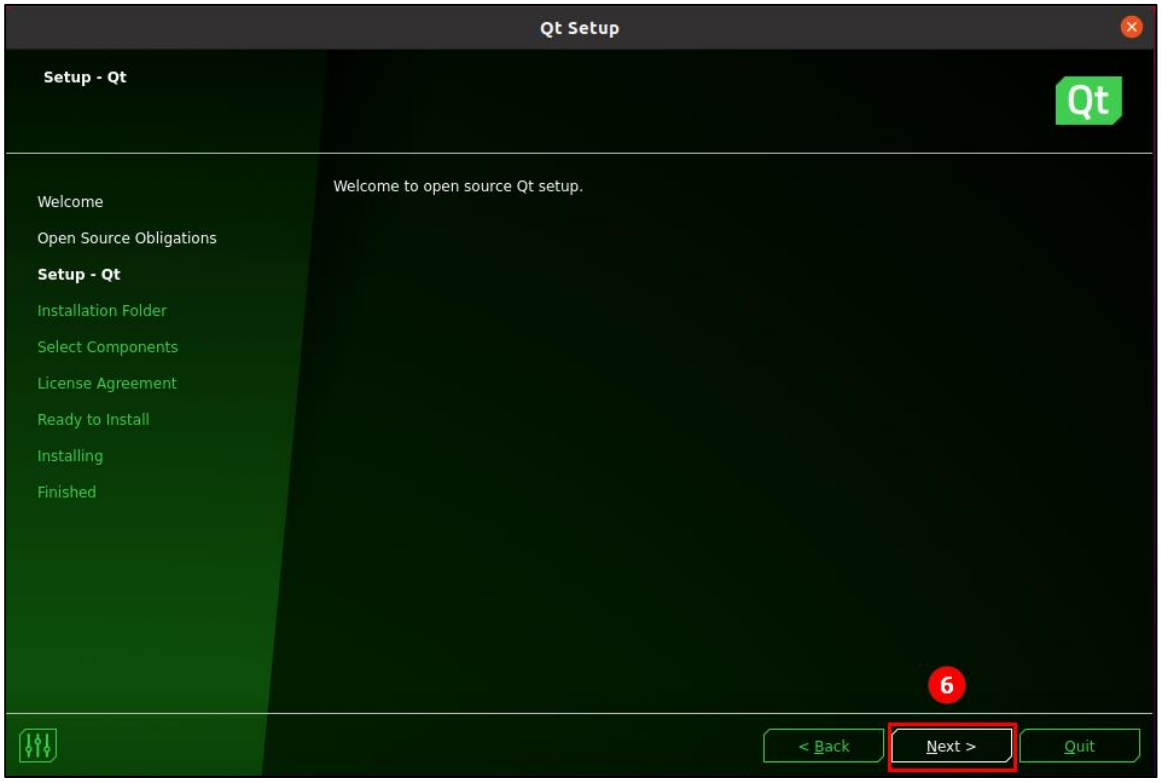
如下图弹出安装界面，请输入你的 Qt 帐号与密码，帐号一般是用邮箱注册的，然后点击下一步。



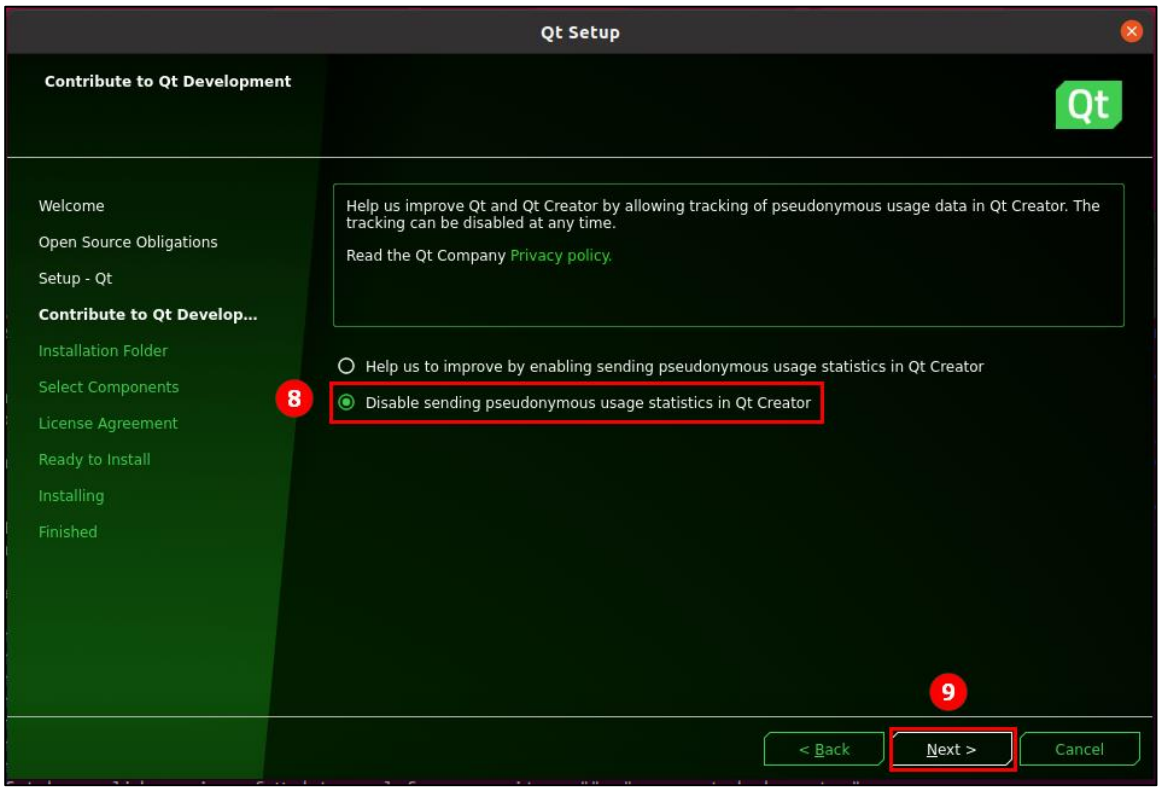
勾选③④，点击下一步。



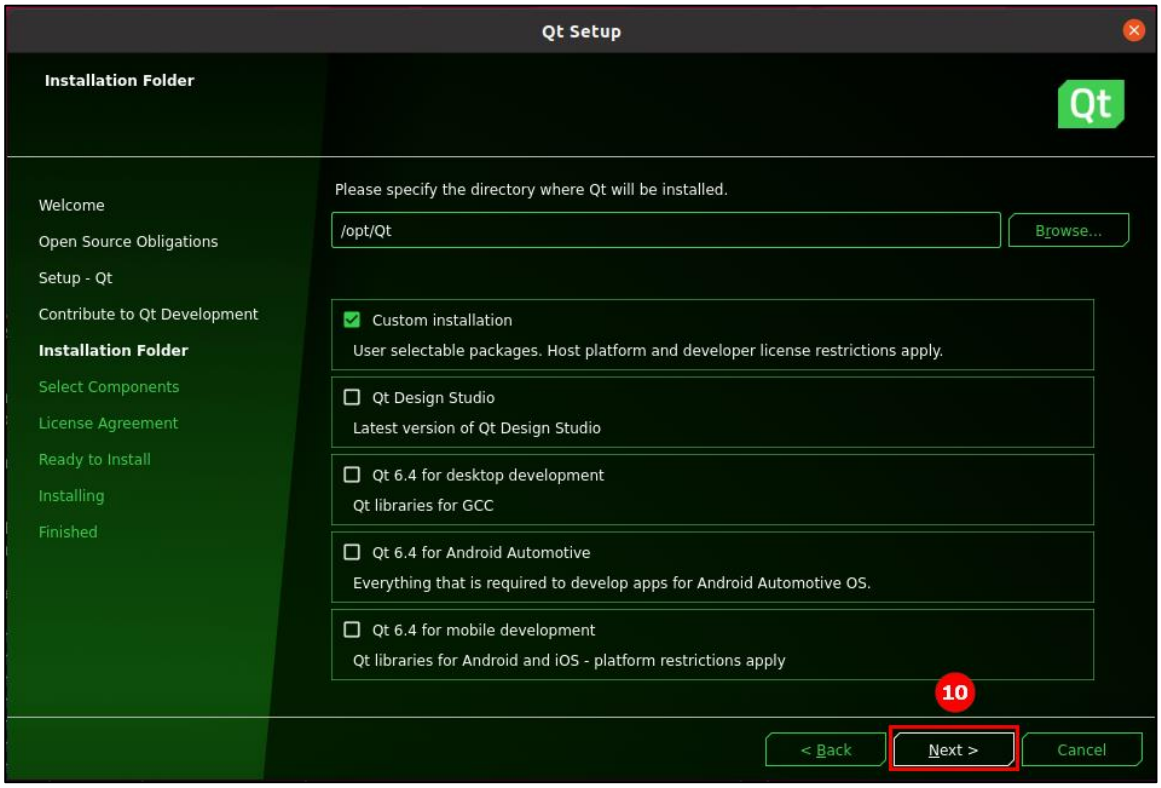
点击下一步。



第⑧处，随便选一个，这是问咱们要不要为 Qt 做点贡献，咱们也想，但是能力不允许。点击下一步。



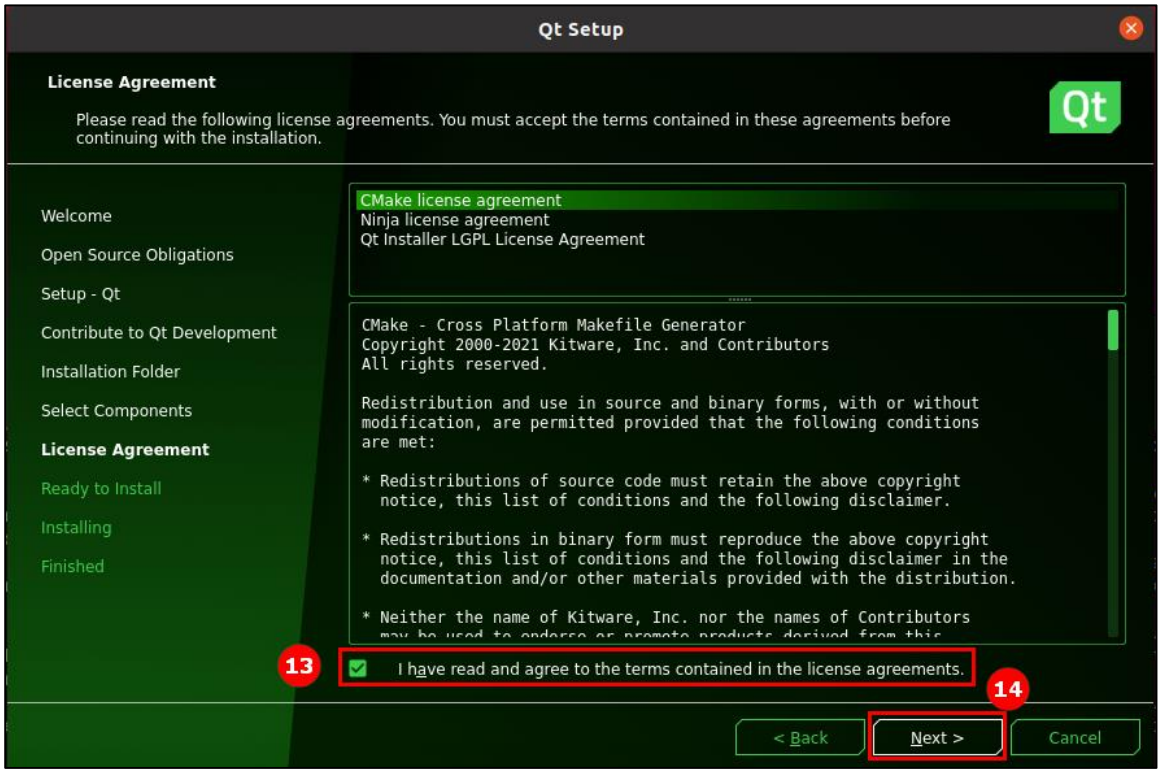
若你使用的是 sudo 权限来安装 Qt，默认会安装在/opt/Qt 这个路径下，建议大家与笔者同一路径，后面让 Qt Creator 支持中文输入，也会用到此路径。点击下一步。



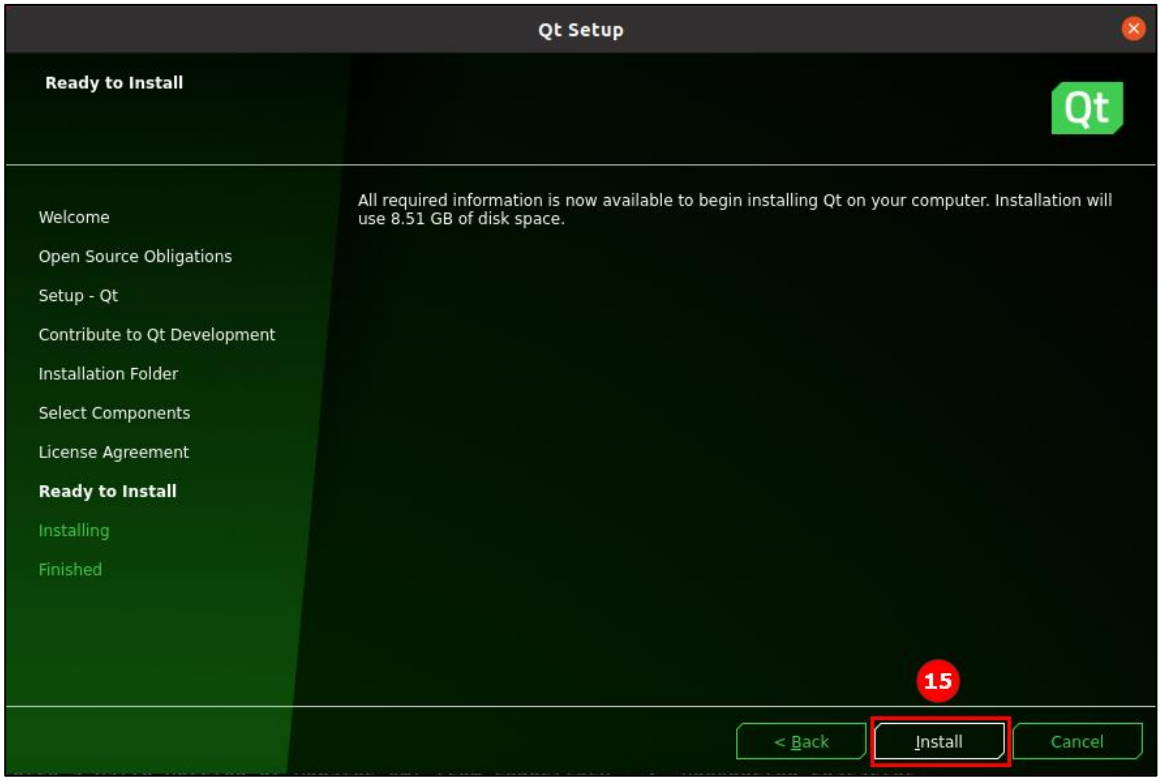
Qt Design Studio 是作为 QML 的 UI 版，当然它也可以编写代码。当前我们用 Qt 开发还是以 Qt Creator 为主。所以我们按如下选择组件来安装，以 RK3568/RK3588 Buildroot 里的 Qt 版本是 Qt5.15.2 和 Qt5.15.8 为例（后面如果有 Qt6 的安装教程都是一样的步骤），但是貌似没提供 Qt5.15.8 的安装选项（但是不清楚后面的 Qt 安装包有没有），但是不要担心，只要大版本号 Qt5.15 相同，版本上的 API 都是可以相互参考的。所以我们需要安装 Qt5.15.2 这个 Qt 版本，这样开发的好处是，同一个 Qt 开发版本语法上可以兼容。勾选 Qt5.15.2，除了安卓选项，如果你想开发安卓，安卓有很成熟的 Android Studio 开发。如果你非要安装安卓编译套件，这将会占用下载时间与大量空间。剩下的根据个人需要安装，笔者建议初学者与笔者一样选择安装。

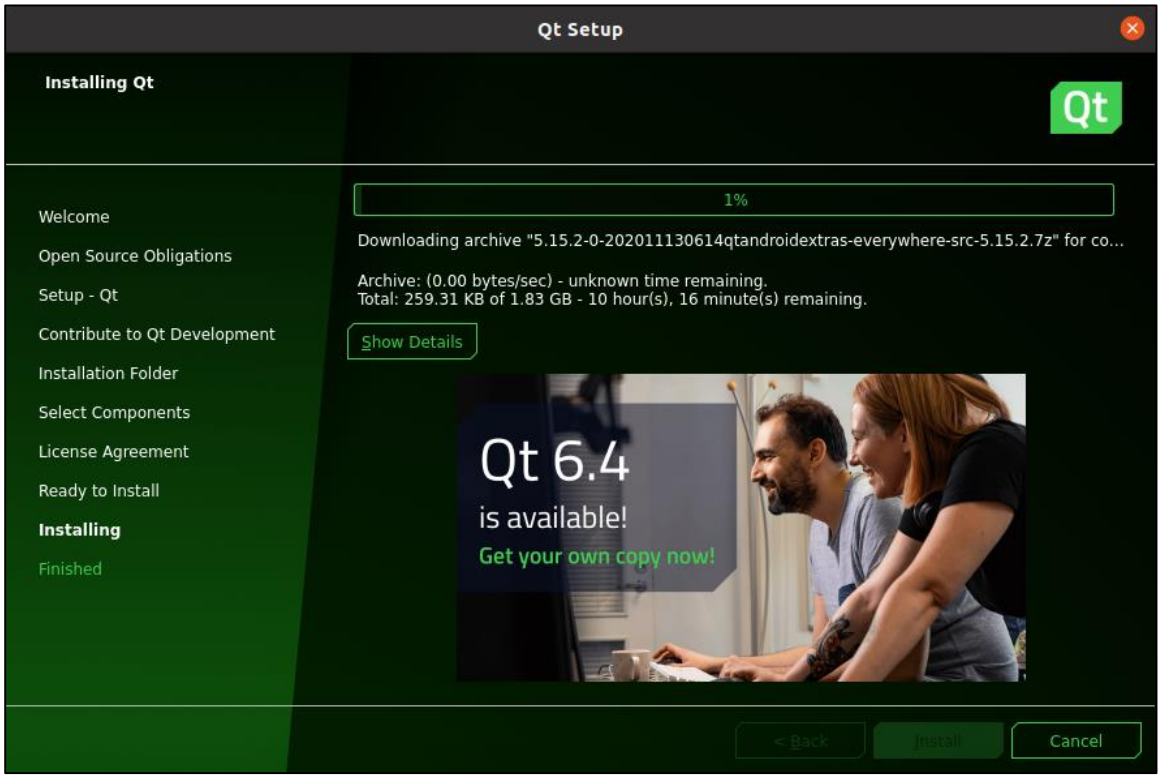


接下来我们同意他里面的协议即可，然后点击下一步。

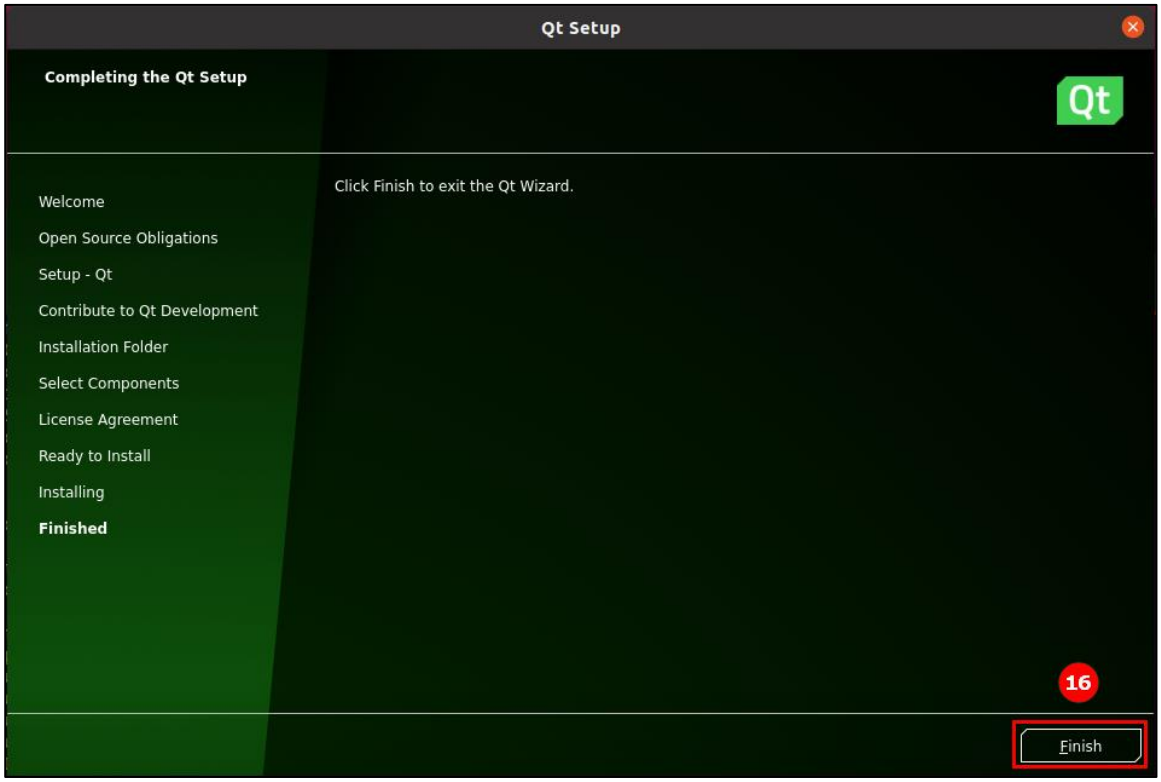


安装大概需要 8 多 GB，点击下一步。





下载速度看服务器和个人网速。

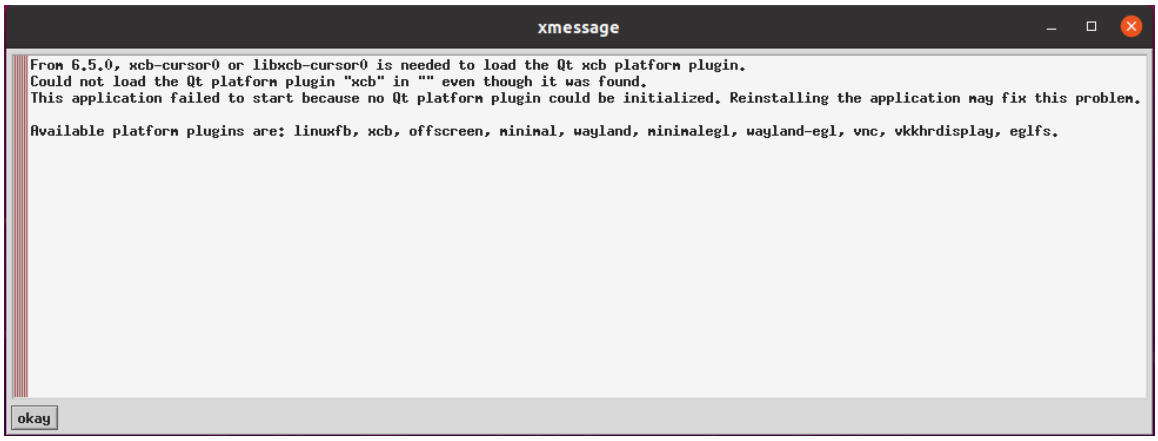


安装完成，在左下角应用程序中心找到 Qt Creator，点击打开 Qt Creator。



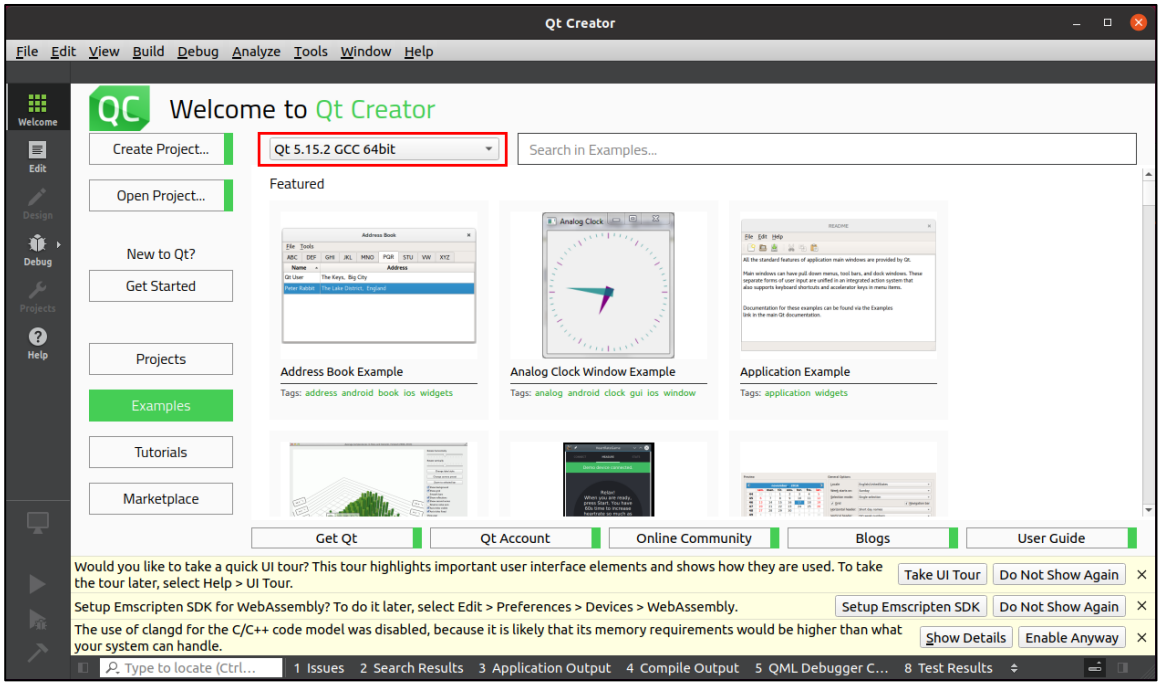
打开 Qt Creator 的界面如下，安装完成。注意，安装完成后请在 QtCreator 里尝试输入中文，如果能输入中文就不用做 1.2 小节。笔者写文档安装的 qtcreator 是 10.0.2，几个月后，官方将它升级到 qtcreator11.0 了，同时也可以直接输入中文了。后面 qtcreator 将会有更新的版本也不一定。所以笔者文档就不改了。

若出现类似下面的错误，或者日后有其他错误。



那么我们根据上面的提示安装相应的库。

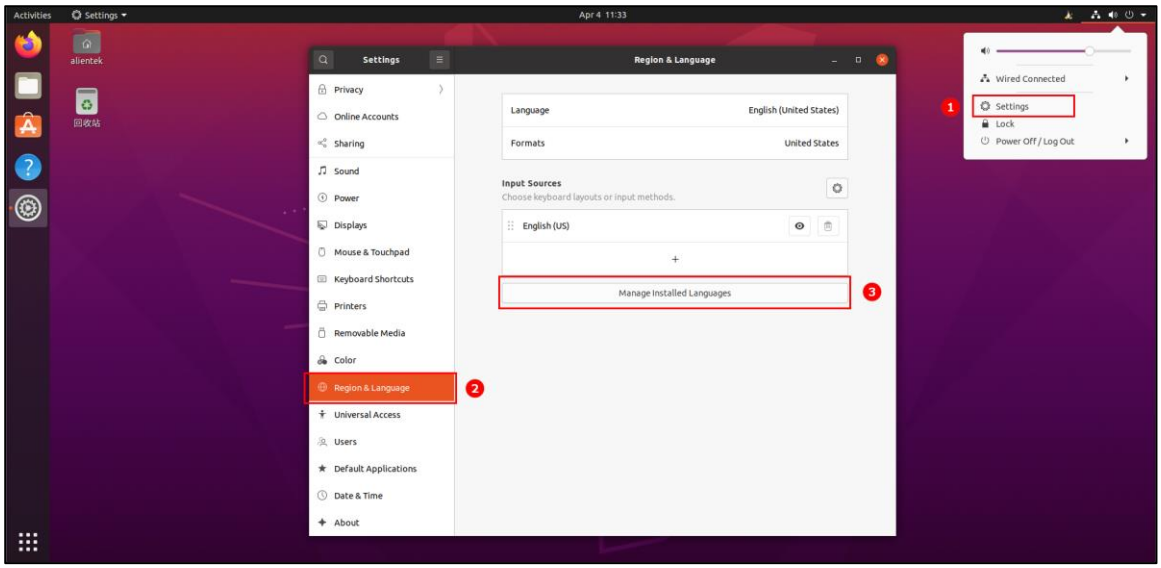
```
sudo apt-get install libxcb-cursor0
```



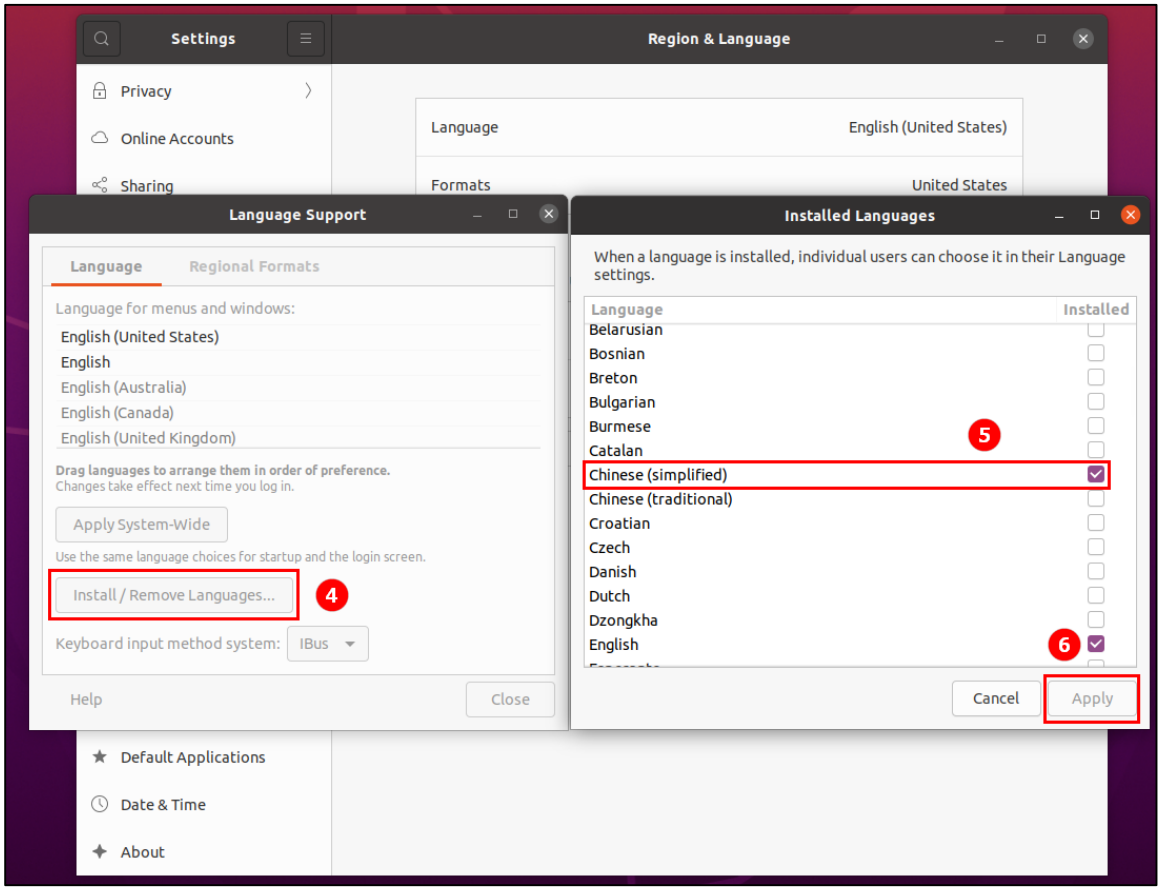
1.2 配置 Qt Creator 输入中文

1.2.1 配置 Ubuntu 中文环境

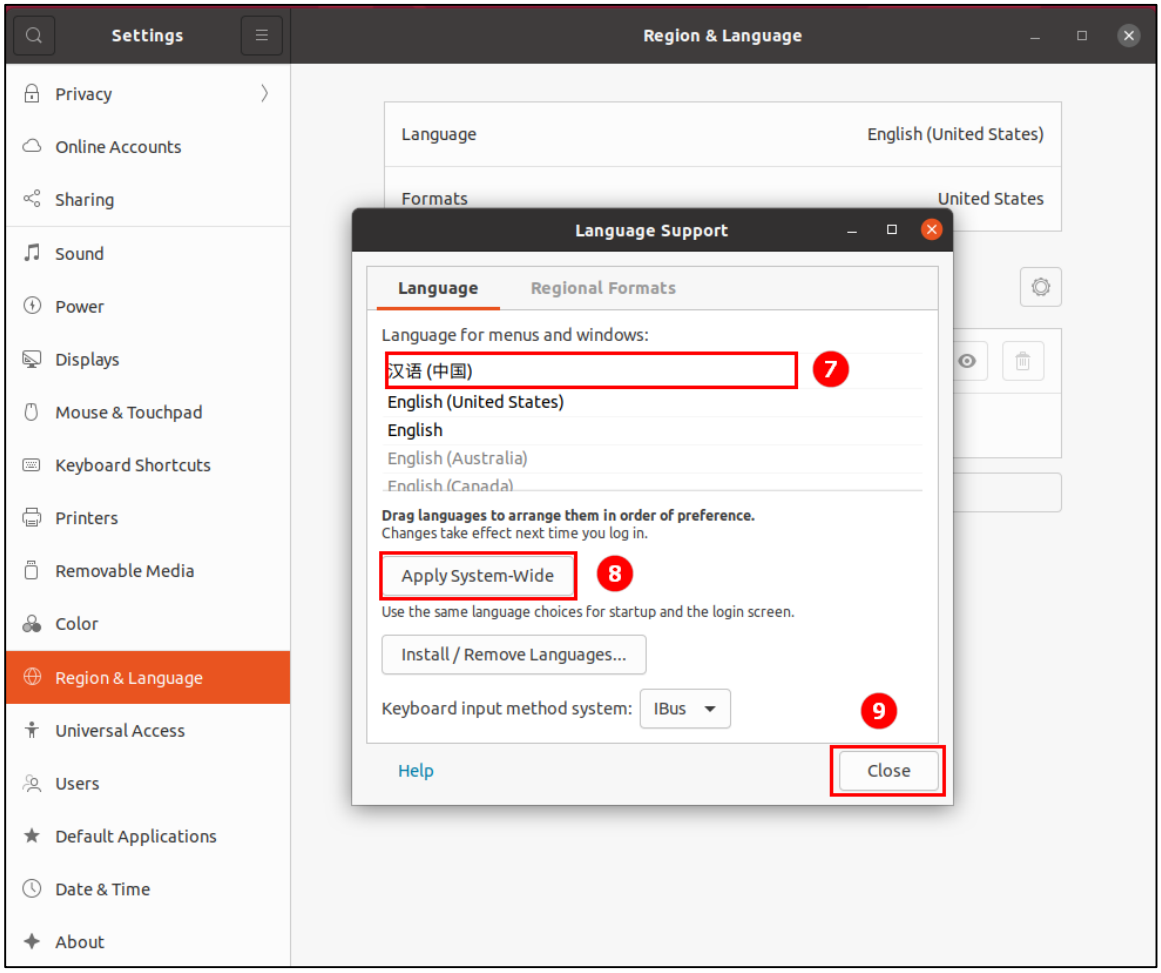
Ubuntu 的默认安装环境是使用的语言是英文，很多同学不习惯。现在我们将 Ubuntu 的系统语言设置成中文。在 Ubuntu 右上角，点击设置图标如下图第①步。



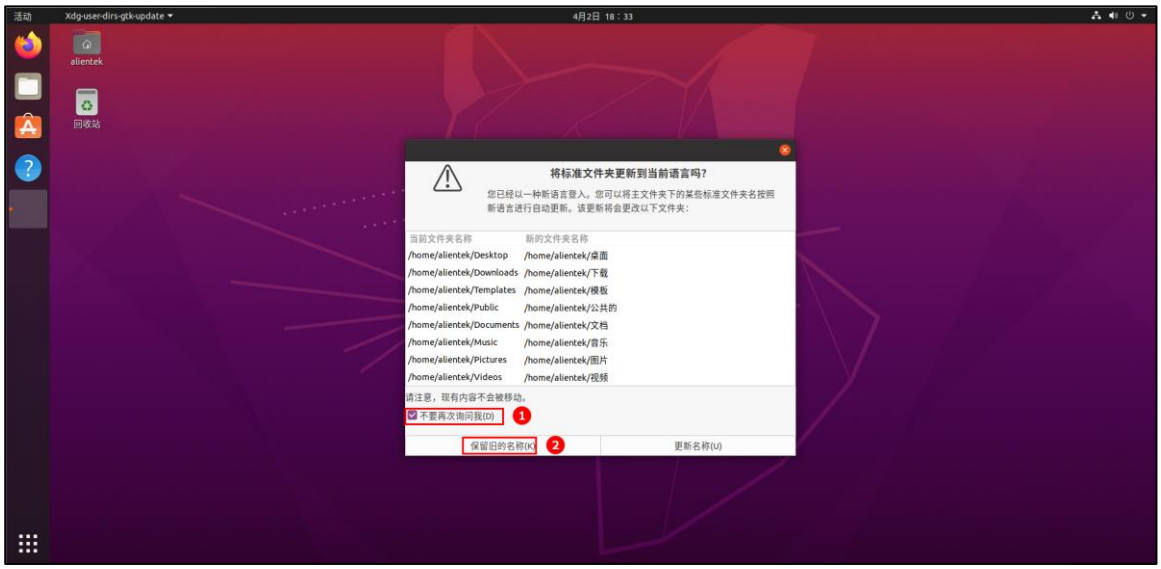
按如下图设置，点击（install/Remove Languages …）安装或者移除语言，在安装语言处选择简体中文，点击 Apply（应用）即可。



第⑦步中，在列表中往下滚动，找到汉语（中国）选项，按住该项将它拖到第一项，可能会比较难拖，多尝试几次，直到汉语（中国）成为第一项。这样首选语言就是中文的了。



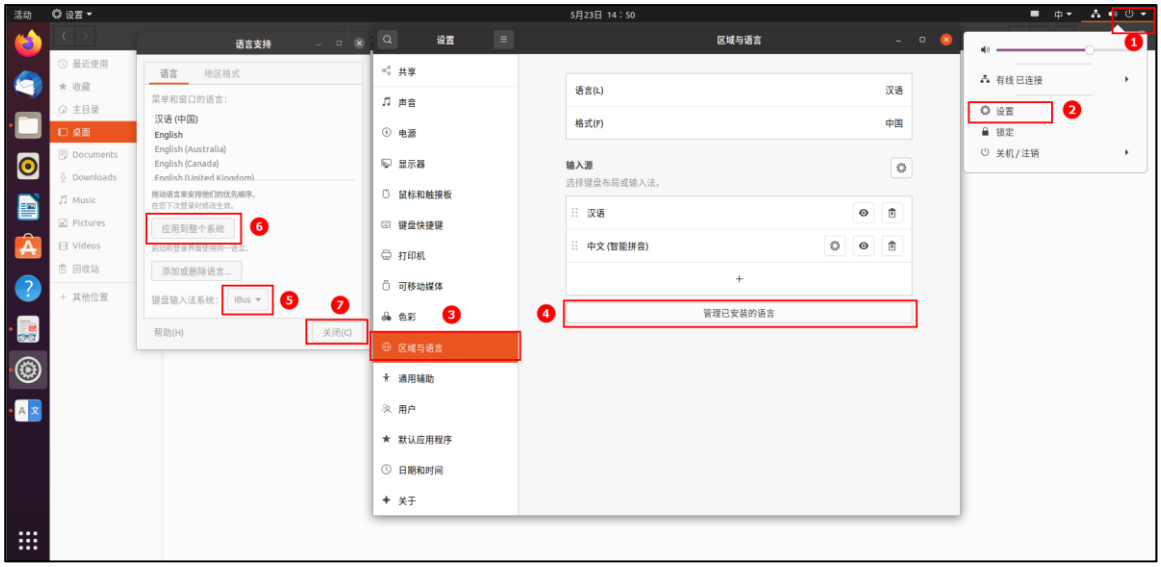
配置完成后，点击重启（或者注销 Ubuntu），重启后，因为我们已经更新了系统的语言，Ubuntu 询问我们需不需要将系统文件夹的名称也改成中文。这里作者选择否，保留旧的名称。保留旧的名称有一定的好处，就是我们进入这些访目录时，直接使用英文，不用切换到中文输入法。严格来说，最好是统一用英文环境开发了。这里为了初学者或者有强迫中文者，所以我们这里需要配置中文的环境，及后期开发 Qt 需要写中文注释。



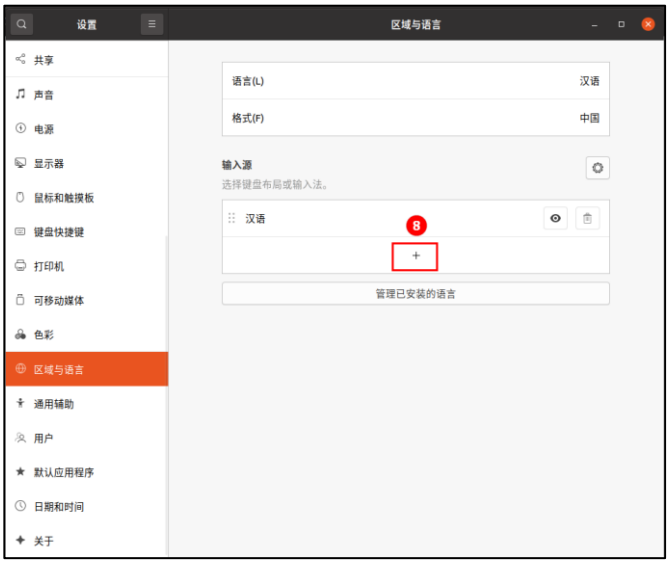
1.2.2 配置中文输入法

在上面我们已经配置好中文环境，并有拼音输入法 ibus，实际上 ibus 输入法是 Qtreator 一直以来都支持的输入法。网上有很多 Ubuntu 安装 Qtreator 输入中文的教程，甚至笔者以前写过 Qtreator fcitx 输入法的教程，虽然可以用，但是却不是最方便最简单的方法。经正点原子用户反馈刚装好的 Ubuntu 却可以在 Qtreator 里输入中文。笔者研究了有些 Ubuntu 却不行，原因是在配置 Ubuntu 中文环境的时候可能有些差异。现在教大家来使用这个 ibus 输入法。

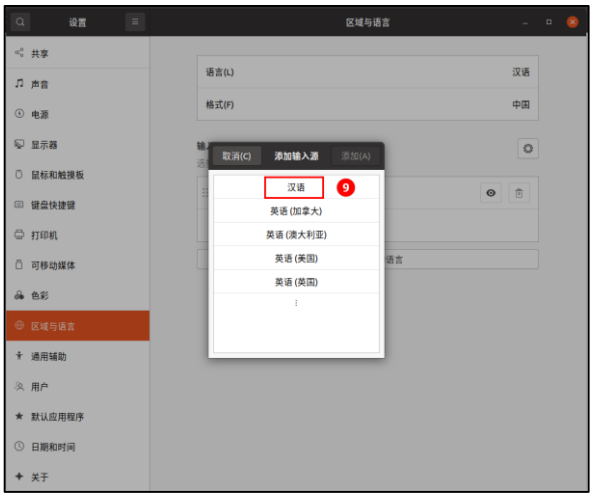
按如下步骤，确认第 5 步中的键盘输入法系统设置为 IBus，Ubuntu 系统默认输入法就是 IBus，如果你没有修改过就不需要修改。



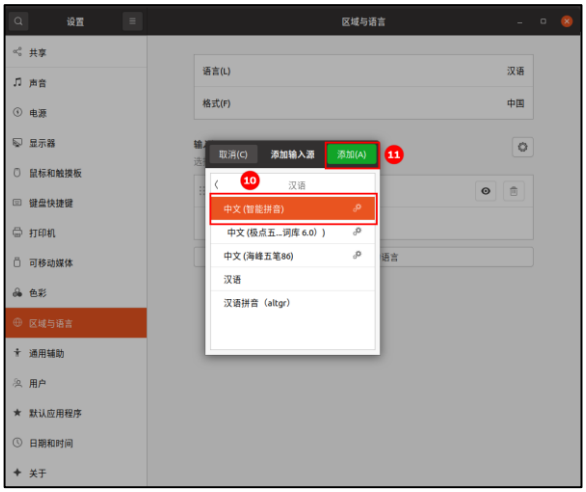
现在回到区域语言处，如下图，在某些 Ubuntu 环境配置的时候可能没有设置系统的输入源，也就是系统要选择何种输入法输入。下面的图中笔者已经将（中文（智能拼音））输入法移除了，现在演示重新加上这个中文（智能拼音）输入法。如果你看到有中文（智能拼音）输入法了，那么你的 Qtreator 理论上可以输入中文了，那么你就无需再添加这个中文（智能拼音）了。



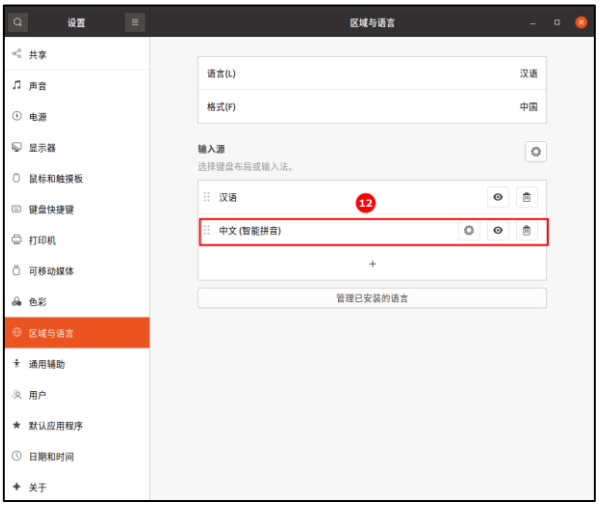
点击添加输入源，选择汉语。



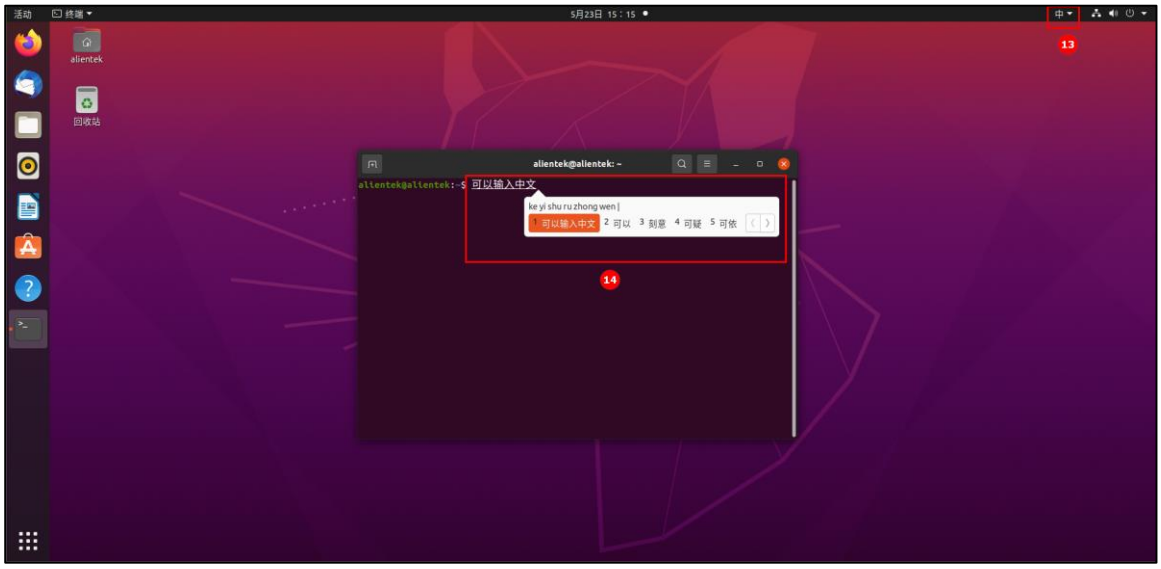
选择中文（智能拼音），大家可以看到下面的这些输入法都是 IBus 输入法，可以根据个人的输入法习惯选择，比如你还需要五笔输入法，那么你就选上下面的带“五笔”输入法的字样的输入法即可。然后再点击添加！



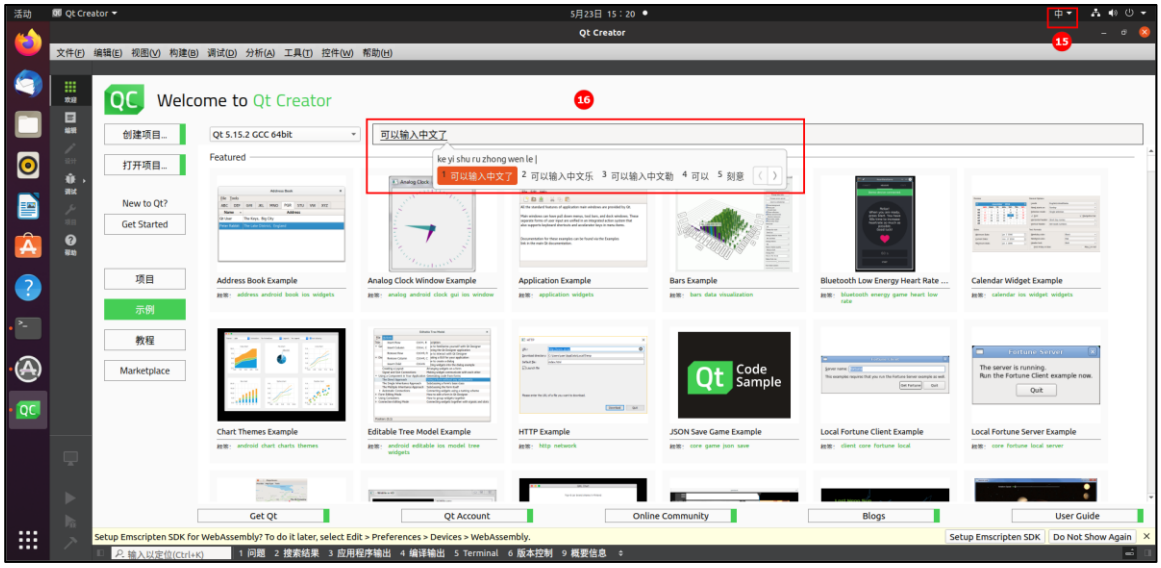
完成如下图，可以看到中文（智能拼音）已经为系统输入源了。现在我们重新启动 QtCreator 或者重启 Ubuntu，让配置的环境生效。笔者建议用户重启 Ubuntu，防止环境未生效。



现在我们在终端里检验 IBus 拼音输入法是否正常使用了。如下图，先在 13 步中，选择中文输入，再在终端里开始输入。成功了！中英文切换只需要按 shift 键切换。



现在我们来打开 QtCreator，如下图，可以使用 IBus 拼音输入法输入中文了！

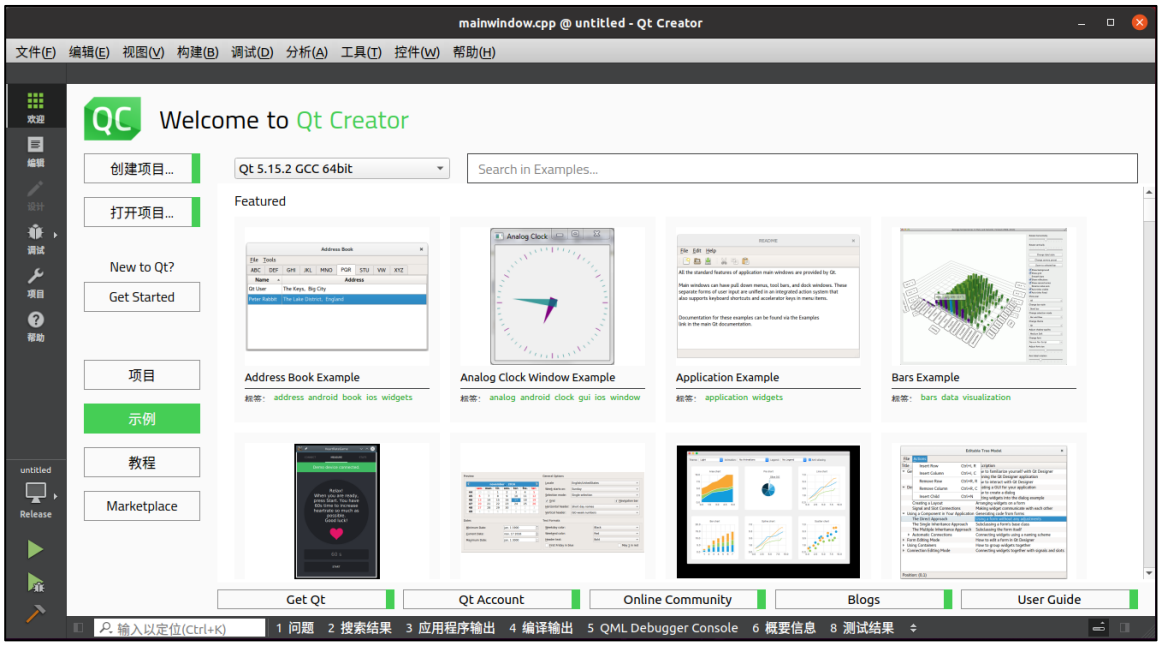


1.3 Qt Creator 简单使用

1.3.1 Qt Creator 界面组成

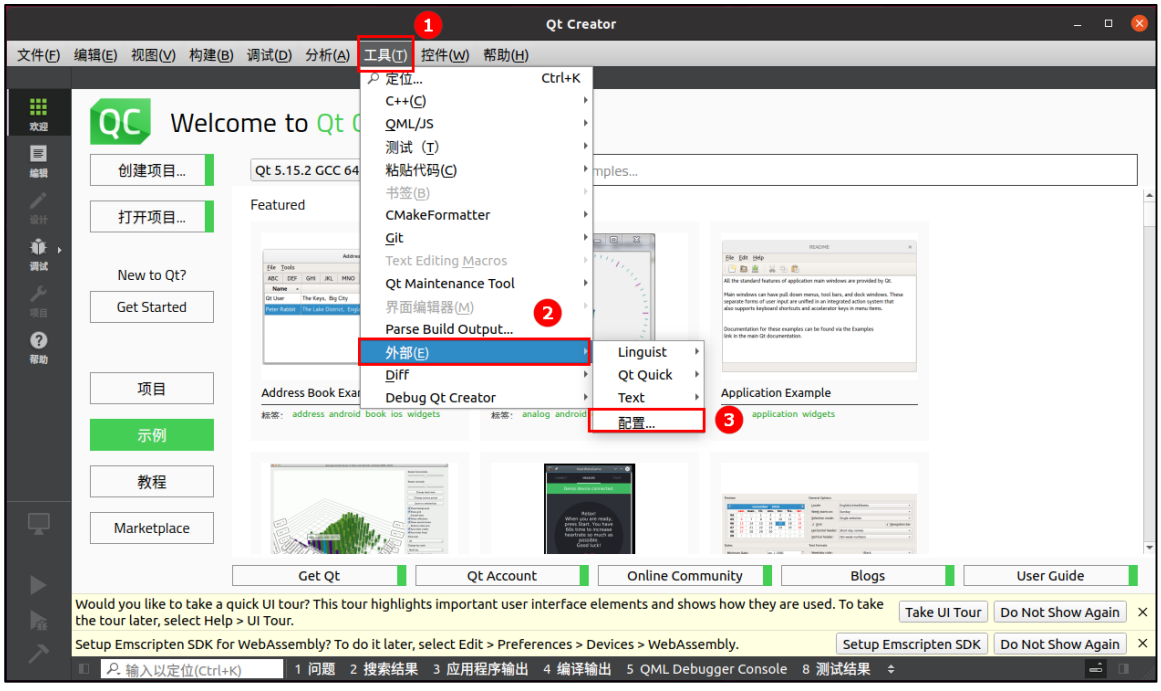
启动 Qt Creator 后，Qt Creator 的主界面如下图，默认打开的是欢迎页面。可以看到 Qt Creator 里自带很多示例。在 Ubuntu 里，由于 Qt Creator 安装在/opt 目录下，这个目录普通用户是没有权限写的，只能够读。如果要打开示例先点击后选择“复制项目并打开”。作者比较喜欢 Qt Creator 界面设计的。十分简洁，还自带示例，不会写也会参考！最重要的是 Qt Creator 里**左侧栏**的“帮助”按钮，有很多使用说明，总结的非常好，对学习 Qt 的类有很大帮助，如果学习到某个方法或者类不会用，可以打开这个“帮助”来搜索这个类或者方法的用法。没有任何一本教程能有 Qt 帮助文档这么详细了，可惜是英文的，初学者学起来还是会一定难度。多多参考这个 Qt 帮助文档，因为那里能学到很多，例子说明也特别多，每个方法、类、信号与槽等都有详细的解

释。



1.3.2 Qt Creator 设置

Qt Creator 里的设置比较重要。在 Qt Creator 的设置里我们可以做些什么呢？我们一般会设置字体的大小、颜色和背景颜色等这样看起来比较炫，或者比较符合自己的风格。Qt Creator 里也有很多主题，拥有好多种搭配，相信有一种是您喜欢的，如果不喜欢可以自定义里面的字体大小、颜色和背景颜色等。点击顶部的菜单栏的“工具”》“外部”》“配置”，这里与 Qt Creator4.12 界面有些差别。



打开选项后可以看到如下图的界面。可以看到默认已经有 Desktop Qt 5.15.2 GCC 64bit 编译

器，这个编译器编译出来的程序可以直接在 Ubuntu 上运行。如果你看到的结果和笔者看到的结果不一样，主要看比如编译器处，已经识别本地 C 和 C++编译器，因为笔者在文档最开头就叫大家安装 g++与 gcc 等一些软件/库了。



下面我们主要介绍常用的几项：

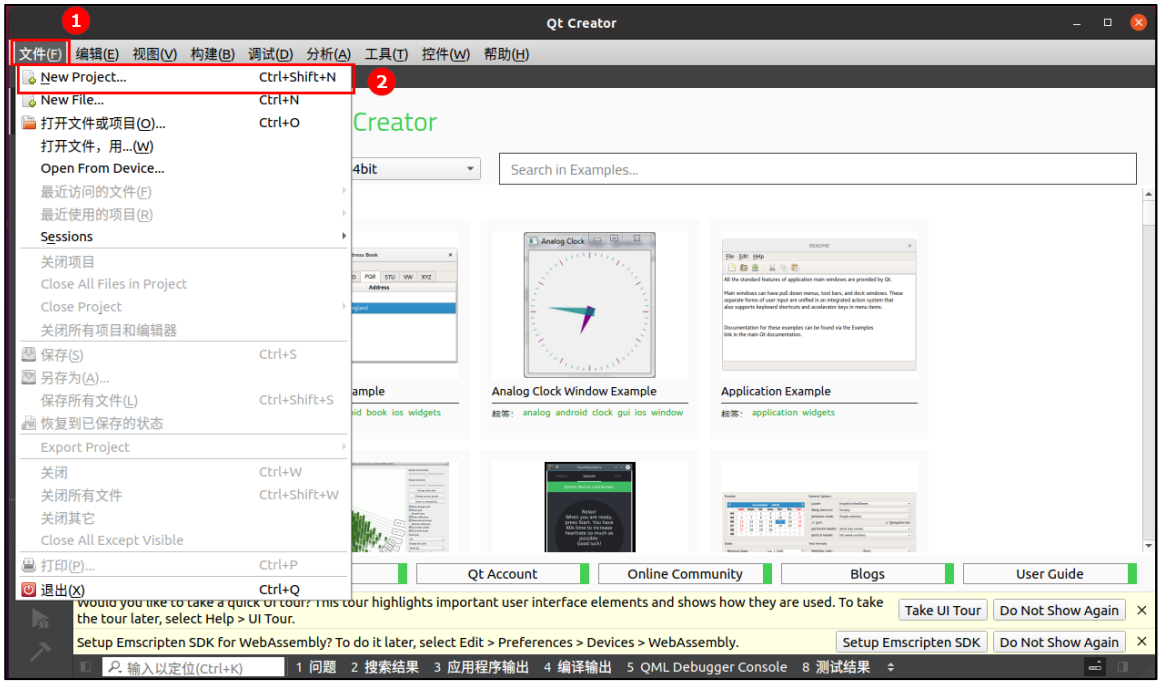
1. Kits：主要显示的是编译工具。我们在 Ubuntu 安装 Qt Creator 时，在安装选项里已经勾选了 Desktop Qt5.15.2GCC 64bit 这个选项。所以在 Kits 这个页面就能检测到安装的编译工具。还有 Qt Versions 等项都可以自由查看。重要的是我们可以在这个 Kits 里配置 ARM 平台的编译工具（后面以某个 ARM 板卡平台为教程时有讲到如何配置），之所以 Qt 能够跨平台，是因为 Qt 有不同平台的编译工具。
2. 环境：在这个项里可以设置不同的主题和语言等。Qt Creator 默认的主题和系统语言即可。
3. 文本编辑器：可以设置文本编辑器的字体大小、颜色等。还可以设置某些类型的字体颜色，如关键字、字符串和注释等。
4. 构建和运行：常用的是设置项目的目录。其他一般不用修改，默认即可。

1.4 第一个 Qt 程序

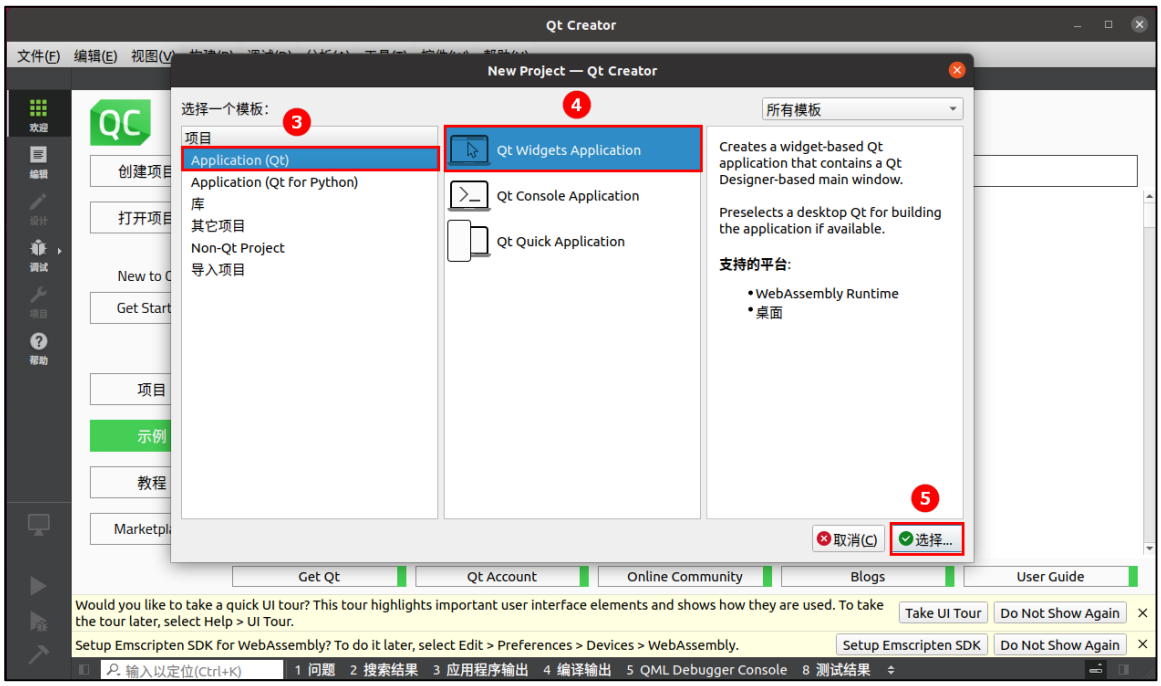
“hello world ”的起源要追溯到 1972 年，贝尔实验室著名研究员 Brian Kernighan 在撰写“B 语言教程与指导(Tutorial Introduction to the Language B)”时初次使用（程序），这是目前已知最早的在计算机著作中将 hello 和 world 一起使用的记录。第一个程序都是以“hello,world”作为默认的第一个程序，我们延续这种经典的做法吧，感受程序的伟大。同时可以让我们初步了解 Qt 项目搭建的基础流程。

1.4.1 新建一个项目

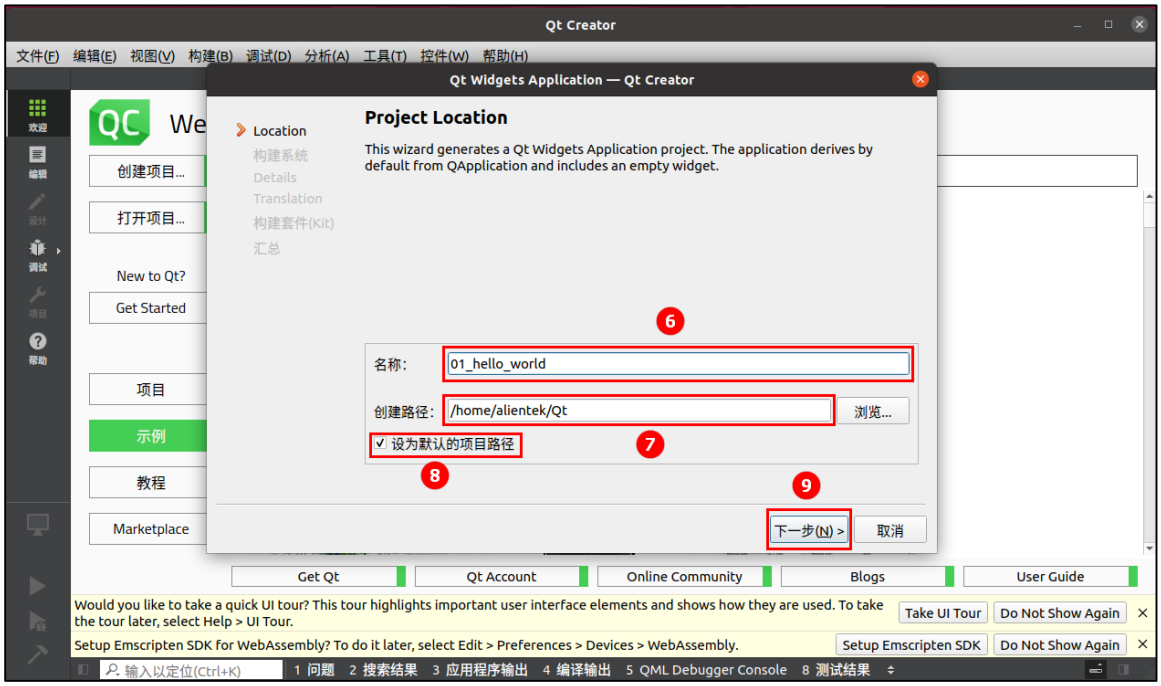
在 Ubuntu20 里打开 Qt Creator，也就是左下角软件中心处点击后，找到 Qt Creator 的图标后点击打开。单击文件 Qt Creator 的文件，选择新建文件或者项目。注意有快捷键 Ctrl＋N。直接在 Qt Creator 激活状态和英文状态的输入法下使用“Ctrl + Shift+ N”也可以快速打开新建项目。



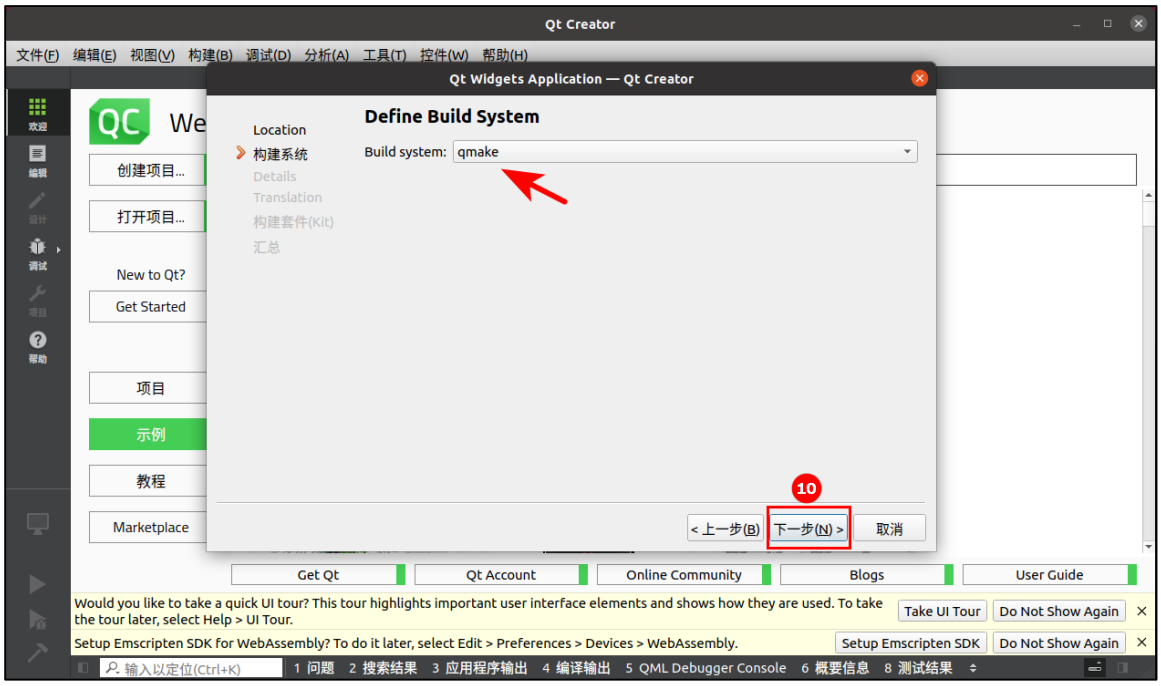
弹出的新建项目如下图，这里我们可以看到有很多模板（包括项目模板和文件和类模板）可以使用，包括 Qt，Qt Quick，Qt for Python，…，C++等等。作为初学者我们选择第一个 Application(Qt)和 Qt Widgets Application，所谓的模板就是 Qt 为了方便开发程序，在新建工程时可以让用户基于一种模板来编写程序，包括 cpp 文件，ui 文件都已经快速的创建，而不用用户手动创建这些文件。这样对用户的开发带来极大的便捷。当然用户可以可以自己手动创建项目，一个一个往里面加也是可以的。但是初学者不建议这么做，我们的目的是先体验一遍项目搭建的流程，等日后有空再自己回头尝试这么做吧！



如果您的输入法现在处于中文输入法，先按 Ctrl 再按 Space（空格）切换成英文输入法，在名称处输入项目为“01_hello_world”，这里的项目路径为笔者个人的家目录路径下的 Qt 目录下，勾选设为默认的项目路径，这样以后做项目实验时都是默认选择这个目录作为项目路径而不用自己手动选择路径了。选择下一步。

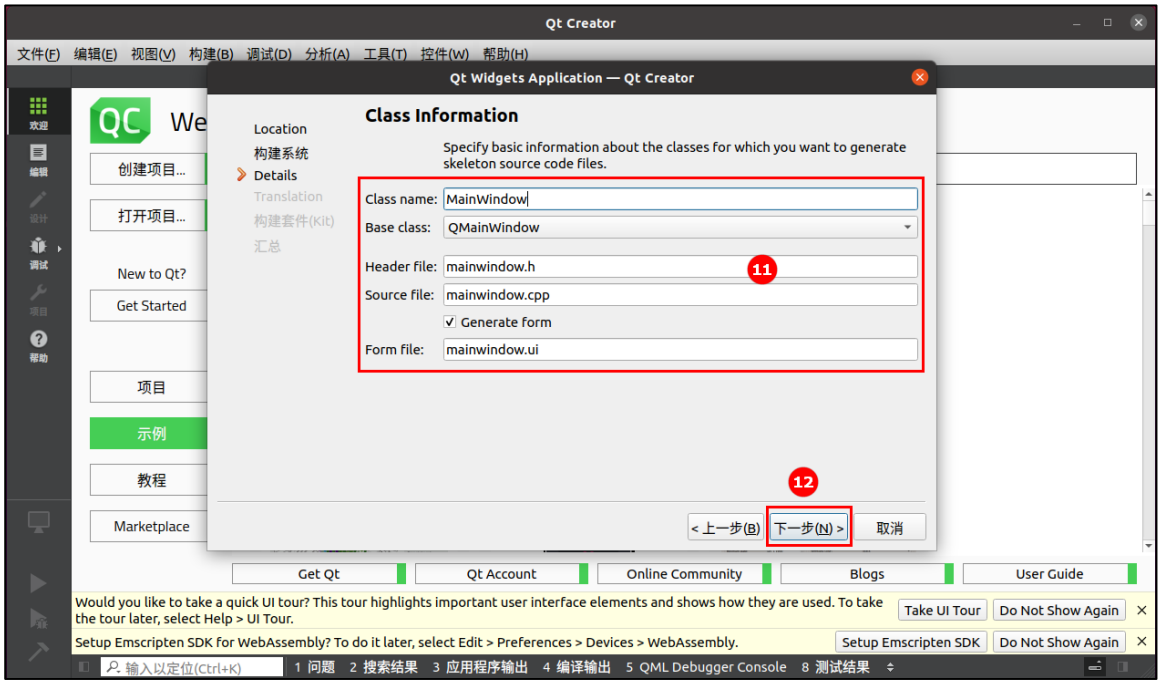


构建系统可选择 CMake，qmake, Qbs 对于 CMake 我们比较熟悉，用于生成 Makefile，那么 qmake 与 Qbs 也同样道理，qmake 与 cmake 比较常用。我们需要选择 qmake。

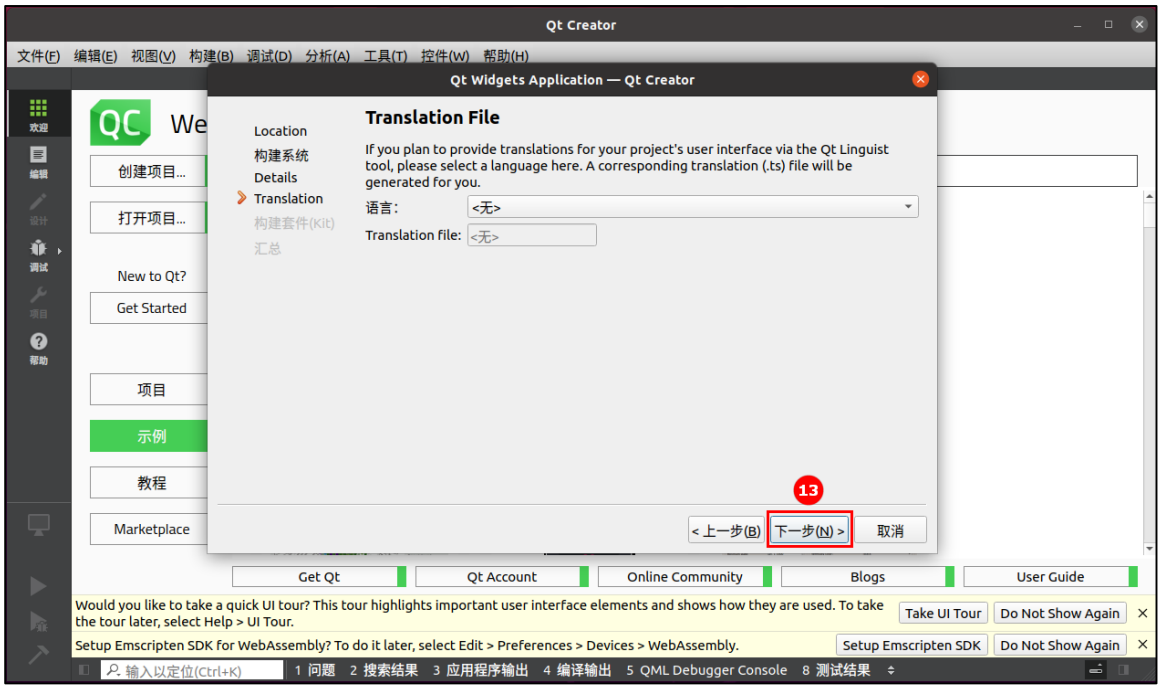


这里默认选择的基类为 QMainWindow。在 Base class 一项中我们还可以看到还有 QWidget 和 QWialog 这样的基类可以选择。在 C++篇我们已经学习什么叫基类，简单的来说，我们创建的这个项目是基于 QMainWindow 类去开发的。默认勾选“Generate form”，意思是生成 ui 窗体文件 mainwindow.ui。为了学习方便，我们统一默认基类为 QMainWindow，但是注意，在嵌入式里一般不需要标题栏，状态栏等，所以常用的是 QWidget 基类。

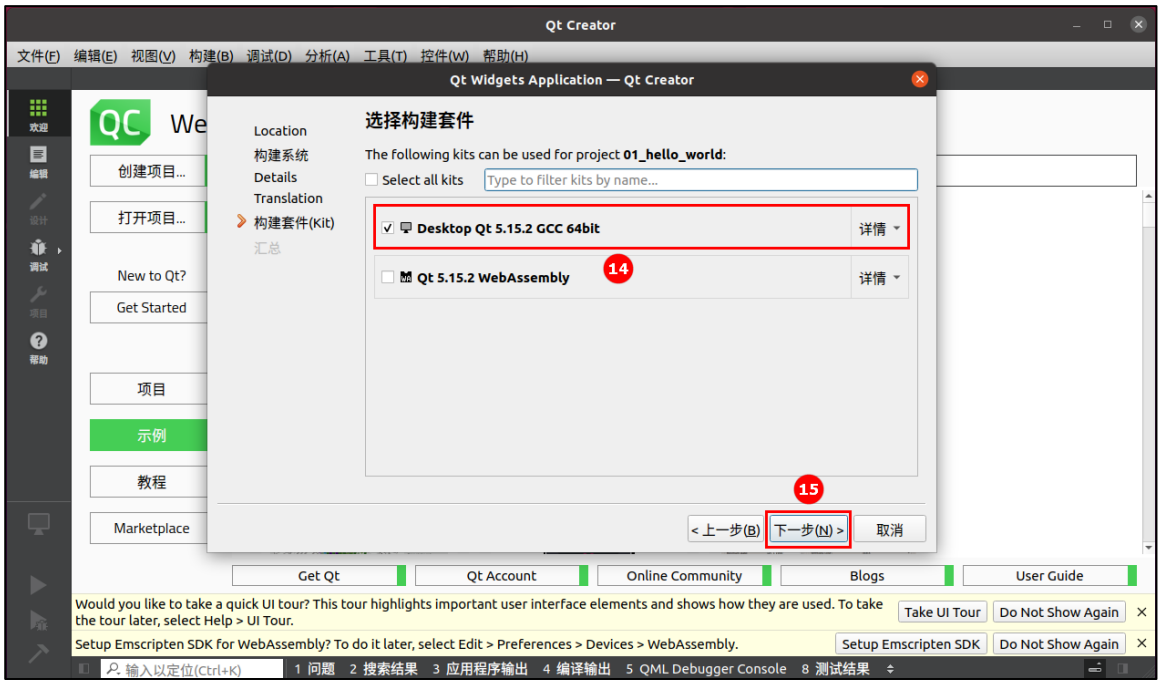
- QMainWindow：主窗口类，主窗口具有主菜单栏、工具栏和状态栏。类似于一般的应用程序的主窗口。如果您想做个嵌套的窗口程序开发的软件，不妨选择这个 QMainWindow。
- QWidget：是可视界面类的基类，也就是说 QMainWindow 类也是由 QWidget 继承封装而来。所以 QWidget 要比 QMainWindow 功能少一些。
- QDialog：对话框类，建立一个对话框界面。比较少使用此项作为基类。一般以 QMainWindow 和 QWidget 作为基类的居多。注因为 QWidget 不带窗口标题栏等，嵌入式里最好 QWidget。



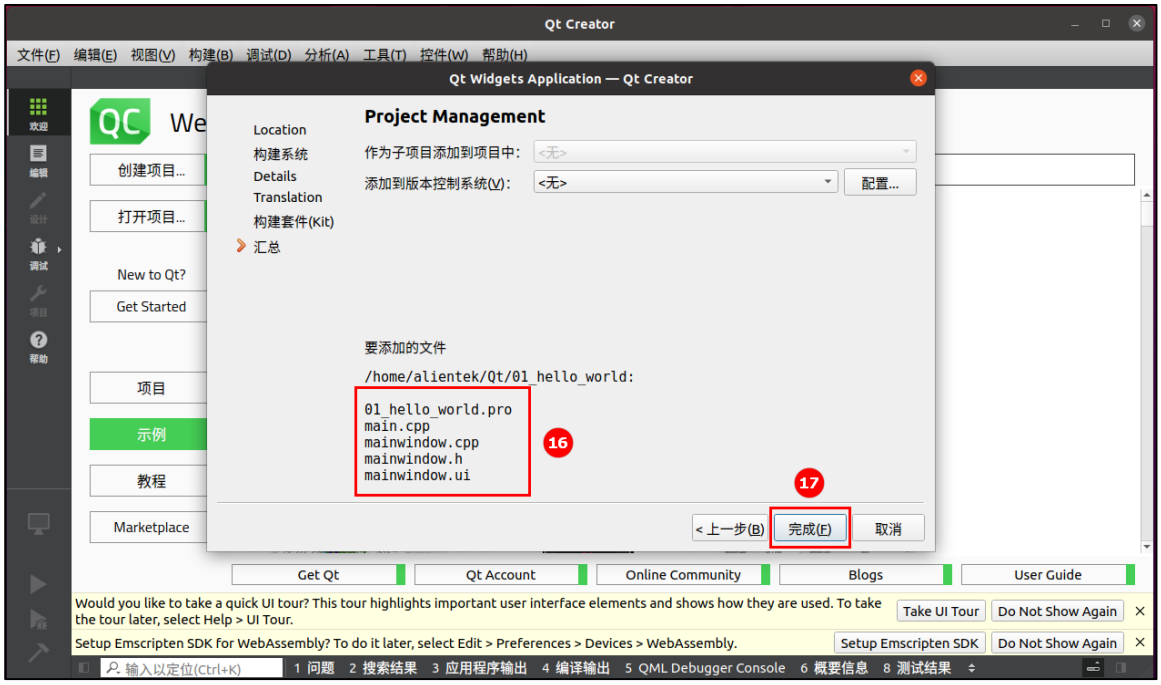
不用选择翻译文件，直接点击下一步。



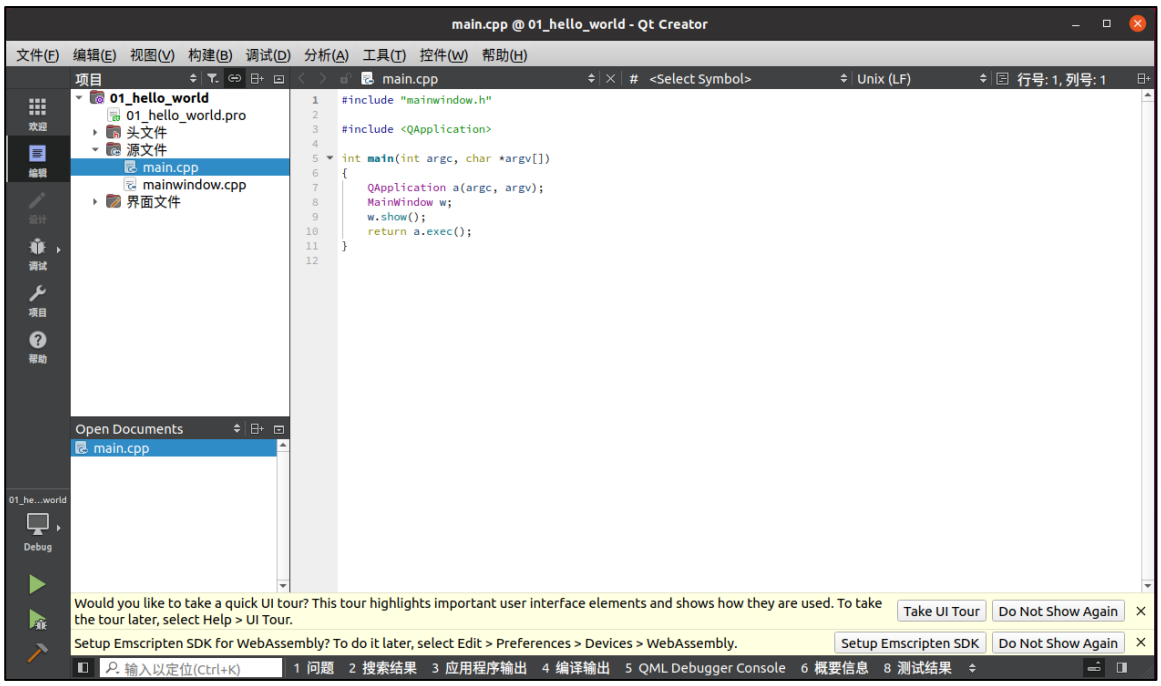
勾选编译器，这个 Desktop Qt5.15.2 GCC 64bit 编译器是我们在安装组件时选择的，使用这个编译器可以编译出 Ubuntu 版本上跑的可执行程序。这么一说是不是觉得还能编译出其他平台的可执行程序？没错，假若我们现在有 ARM 平台的 Qt 编译器（第二章有讲到），那么选择 ARM 平台的 Qt 编译器即可编译出 Qt 在 ARM 平台上的可执行文件（这里说的可执行文件类似 window 的 exe 程序文件一样，直接能够运行）。点击下一步。另外一个可能是最近出来的编译器，笔者也没有用过，看样子是与 web 相关，笔者不涉猎。



到这里，左下角可以看到你要创建的项目文件，在这晨 Qt Creator 询问是否使用版本控制，如果您学习过像 Git 这样的版本控制工具，可以选择 Git，将您的项目使用版本控制。默认选择是无版本控制，直接点击完成即可。



新建的“01_hello_world”项目如下。



1.4.2 项目文件介绍

其中，左侧有上下两个子窗口，上面的窗口显示了项目的文件结构，显示当前的项目为“01_hello_world”，细心的还会发现“01_hello_world”是用粗体黑色标明。说明此项目是活动项目，活动项目的项目根节点都是用粗体字体表示的。如果打开了多个项目，那么我们只需要观察哪个是加粗的项目名就表示当前活动项目。

Qt Creator 和其他 IDE 开发软件一样。都是分组管理项目内的各种源文件，下面是项目内的文件简介。

- 01_hello_world.pro 是项目管理文件，这个项目管理文件十分重要，当您加入了文件或者删除了文件，Qt Creator 会自动修改这个*.pro 文件。有时候需要打开这个*.pro 文件添加我们的设置项。
- Header 分组，这个节点下存放的是项目内所有的头文件*.h。
- Source 分组，这个节点下存放的是项目内的所有 C++源码文件*.cpp。
- Forms 分组，这个节点下是存放项目内所有界面文件*.ui。*.ui 文件由 XML 语言描述组成，编译时会生成相应的 cpp 文件，这样交叉编译器就可以编译它了。

1.4.2.1 项目文件*.pro

01_hello_world.pro 文件如下所示。此 pro 文件在高版本的 Qt 与低版本的 Qt 文件有些区别，且在 Windows 与 Linux 系统下生成的 pro 也有些区别，但是变化不大。

01_hello_world.pro 文件内容

```
1  QT       += core gui
2
3  greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
4
5  CONFIG += c++17
6
7  # You can make your code fail to compile if it uses deprecated APIs.
8  # In order to do so, uncomment the following line.
9  #DEFINES += QT_DISABLE_DEPRECATED_BEFORE=0x060000    # disables all the APIs deprecated before Qt 6.0.0
10
11 SOURCES += \
12     main.cpp \
13     mainwindow.cpp
14
15 HEADERS += \
16     mainwindow.h
17
```

```
18  FORMS += \
19          mainwindow.ui
20
21  # Default rules for deployment.
22  qnx: target.path = /tmp/$${TARGET}/bin
23  else: unix:!android: target.path = /opt/$${TARGET}/bin
24  !isEmpty(target.path): INSTALLS += target
```

第 1 行，添加了 Qt 的支持的模块，core 与 gui 库是 Qt 的默认设置。

第 3 行，比较 Qt5 版本，如果是 Qt5 版本，在 main.cpp 中 application 是在 QtWidgets 中的，因此要包含这个库。

第 5 行，Qt5.15 或者 Qt Creator10.0.0 使用的是 C++17，较低版本使用是 C++11。

第 11 行，SOURCES 下的是源文件。

第 15 行，HEADERS 下是头文件。

第 18 行，FORMS 下是 ui 界面文件。

第 21 行，部署默认的规则。

第 22 行，qnx:判断是不是 qnx 操作系统，赋值 target.path = /temp/$${TARGET}/bin。

第 23 行，如果是 unix 系统但不是安卓，赋值 target.path = /opt/$${TARGET}/bin。

第 24 行，如果 target.path 为空目录，赋值 INSTALLS += target。

如果需要修改生成目标的可执行程序名字，可赋值 TARGET = xxx。否则 TARGET 将默认取值为项目的名字。

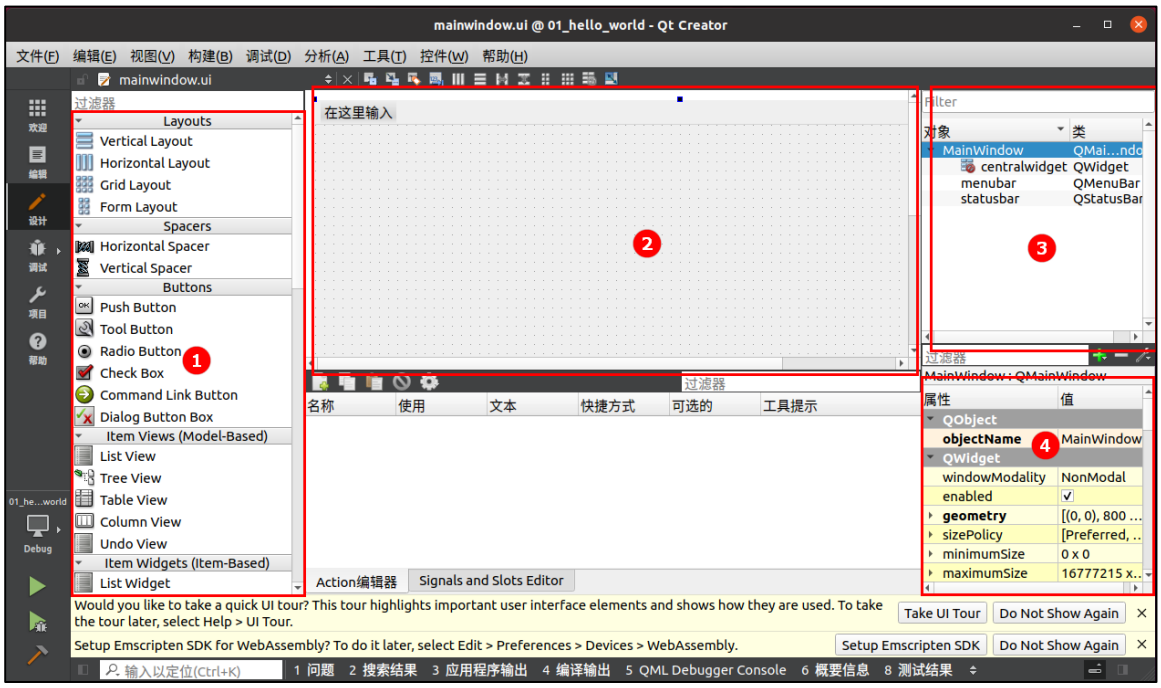
1.4.2.2 样式文件*.ui

mainwindow.ui 是一个 xml 类型的文件，它的 xml 内容如下。这个文件是生成的不能手动编辑。只能够通过图形界面修改其属性。

mainwindow.ui 文件内容

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <ui version="4.0">
3   <class>MainWindow</class>
4   <widget class="QMainWindow" name="MainWindow">
5    <property name="geometry">
6     <rect>
7      <x>0</x>
8      <y>0</y>
9      <width>800</width>
10     <height>600</height>
11    </rect>
12   </property>
13   <property name="windowTitle">
14    <string>MainWindow</string>
15   </property>
16   <widget class="QWidget" name="centralwidget"/>
17   <widget class="QMenuBar" name="menubar">
18    <property name="geometry">
19     <rect>
20      <x>0</x>
21      <y>0</y>
22      <width>800</width>
23      <height>28</height>
24     </rect>
25    </property>
26   </widget>
27   <widget class="QStatusBar" name="statusbar"/>
28  </widget>
29  <resources/>
30  <connections/>
31 </ui
```

双击 mainwindow.ui 后可以跳转到设计界面，如下图。下面主要介绍主体部分。



1. ①是控件栏，有各种各样的控件，上方的 Filter 是过滤器，输入首写字母就可以快速定到我们想要找的控件。
2. ②显示的是我们的窗口程序了，上面已经带有 MainWindow 对象及其几个子对象，默认 MainWindow 就带有菜单栏和状态栏。
3. ③是对象栏，②处用到的对象都在③处显示。
4. ④是属性栏，点击③处对象栏的某个对象，就可以在④属性栏里编辑它的属性了。属性项有很多，包括位置，大小，文字，颜色，字体等等。

1.4.2.3 头文件*.h

点击左边栏的编辑，回到项目的编辑工作窗口。点击项目下的 Headers 下的 mainwindow.h，mainwindow.h 一般有与之对应的一个 cpp 文件叫 mianwindow.cpp。其中 mainwindow.h 包含类的声明，mianwindow.cpp 包含类的实现。这与我们前面学习的 c++是一样的。

mainwindow.h 的内容如下。

mianwindow.h 定义如下

```
1  #ifndef MAINWINDOW_H
2  #define MAINWINDOW_H
3
4  #include <QMainWindow>
5
6  QT_BEGIN_NAMESPACE
7  namespace Ui { class MainWindow; }
```

```
8   QT_END_NAMESPACE
9
10  class MainWindow : public QMainWindow
11  {
12      Q_OBJECT
13
14  public:
15      MainWindow(QWidget *parent = nullptr);
16      ~MainWindow();
17
18  private:
19      Ui::MainWindow *ui;
20  };
21  #endif // MAINWINDOW_
```

第 7 行，定义名称空间 Ui ，里面有一个类 MainWindow ，这个 MainWindow 和第 10 行里的 MainWindow 不是同一个对象。实际上，这个文件里的 MainWindow 类有一个成员*ui 就是指向第 19 行的 Ui::MainWindow 的指针。这都是为了使用.ui 文件设计界面的。

第 12 行，MainWindow 的声明中第一行是 Q_OBJECT，这是一个宏，由 Qt 进行处理，这也是 Qt 针对 C++扩展的地方，所有用到信号的类都要加这个宏。

1.4.2.4 源文件*.cpp

点击项目下的 Sources 下的 mainwindow.cpp。可以看到它的内容如下。

mianwindow.cpp 实现如下

```
1   #include "mainwindow.h"
2   #include "ui_mainwindow.h"
3
4   MainWindow::MainWindow(QWidget *parent)
5       : QMainWindow(parent)
6       , ui(new Ui::MainWindow)
7   {
8       ui->setupUi(this);
9   }
10
11  MainWindow::~MainWindow()
12  {
13      delete ui;
14  }
```

第 2 行，MainWindow 的实现类中，第 2 行 include 了一个文件 ui_mainwindow.h 这个文件是 Qt 根据.ui 文件自动生成的，也就是说 ui_mainwindow.h 要点击编构建后才生成，我们才能看到这个文件。构建/编译后可以在 debug/release 的目录找到这个文件。

第 6 行，在 MainWindow 构造函数中用“,”隔开，new 一个 Ui 中的 MainWindow。这里是一种初始化成员的方法。

第 8 行，ui->setupUi(this);这句话是进行界面初始化，将 this（指的是 MainWindow 类的本身），作为参数传到 setupUi 里，ui 界面文件的对象会以 this 为父对象，所有子对象都将显示在 MainWindow 里。我们要想使用 ui 里的对象，必须将代码写在 ui->setupUi(this)这句话之后，因为 ui->setupUi(this)会先初始化里面的对象，只有初始化里面的对象我们才能使用这个对象。

第 13 行，析构函数里 delete 掉 ui。在 Qt 里我们需要在析构函数里 delete 的对象一般是 new 创建的并且没有父对象的对象。

小提示：this 在成员函数的开始执行前构造的，在成员的执行结束后清除。

点击项目下的 Sources 下的 main.cpp。可以看到它的内容如下。

mian.cpp 实现如下

```
1  #include "mainwindow.h"
2
3  #include <QApplication>
4
5  int main(int argc, char *argv[])
6  {
7      QApplication a(argc, argv);
8      MainWindow w;
9      w.show();
10     return a.exec();
11 }
```

第 3 行，包含 QApplication 类的定义。在每一个使用 Qt 的应用程序中都必须使用一个 QApplication 对象。QApplication 管理了各种各样的应用程序的广泛资源，比如默认的字体和光标。

第 5 行，main()函数是程序的入口。几乎在使用 Qt 的所有情况下，main()只需要在把控制转交给 Qt 库之前执行一些初始化，然后 Qt 库通过事件来向程序告知用户的行为。argc 是命令行变量的数量，argv 是命令行变量的数组。

第 7 行，a 是这个程序的 QApplication。它在这里被创建并且处理这些命令行变量。

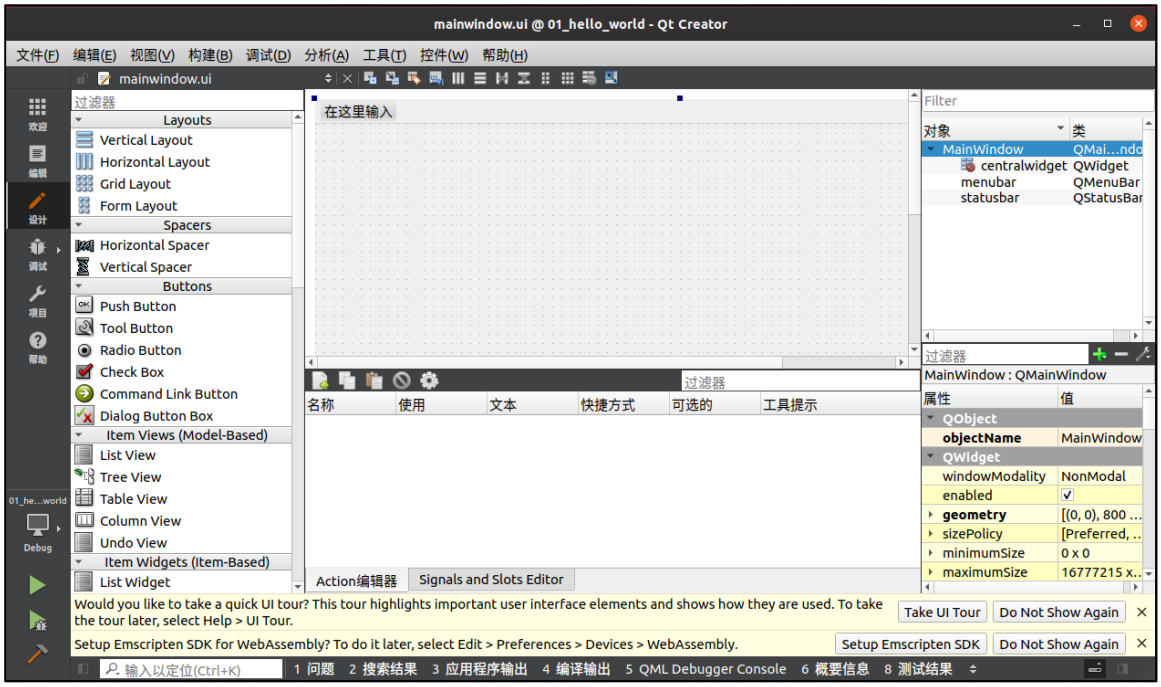
第 8 行，创建一个对象 w，这个对象就是 MainWindow。

第 9 行，调用方法 show()。这样程序界面才能显示。

第 10 行，这里就是 main()把控制转交给 Qt，并且当应用程序退出的时候 exec()就会返回。在 exec()中，Qt 接受并处理用户和系统的事件并且把它们传递给适当的窗口部件。

1.4.3 修改 ui 文件显示 hello world

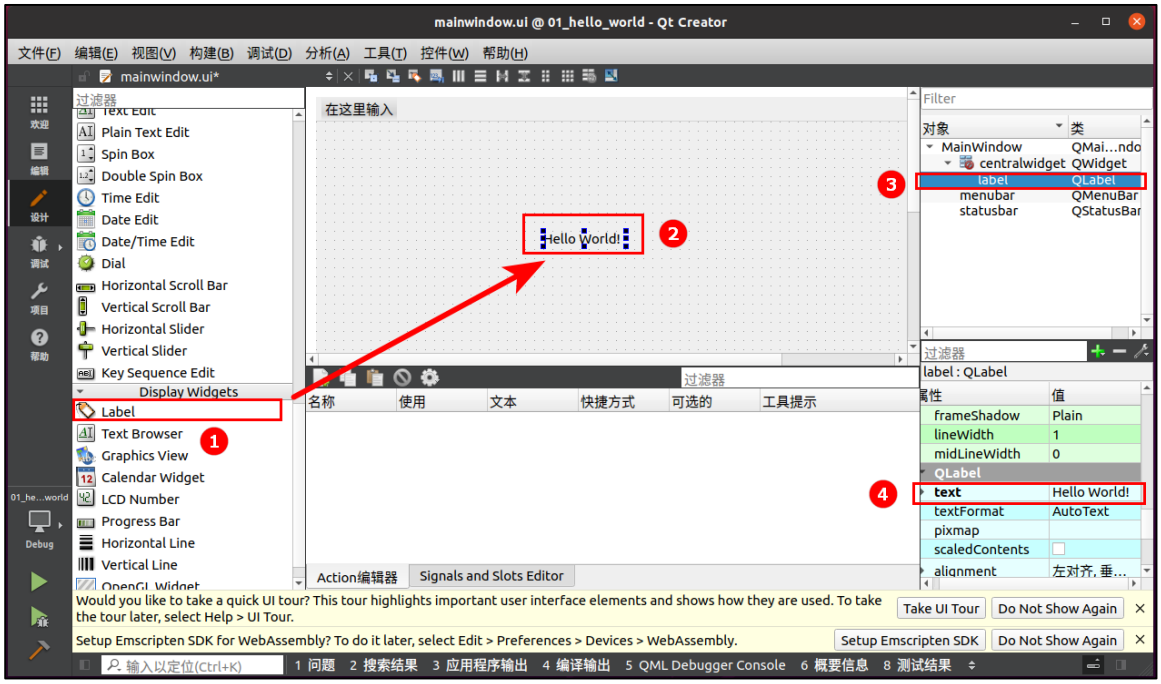
双击 mainwindow.ui，进入“Ui 设计器”页面如下。



可以看到中间的要设计的主窗体目前是空的，但是窗体上已经有好几个成员了，可以看到右边的对象栏，已经有 3 个对象成员在里面了，只是我们看不出来而已，效果不太明显。

我们要在窗体里显示“Hello World!”，那么需要使用左边的控件栏，目前左边的控件栏的控件有很多，也分很多类别。这些对于我们初学者来说，暂时不用知道它们是怎么用的。后面教程其中的类别进行说明它们是如何使用的。现在我们主要是想要显示“Hello World!”。要明白我们的主要目的，切勿看到左边的控件栏就想一个个的去试，又不知道怎么去用。这样的学习方式是不对的。要显示“Hello World!”，那么我们需要用常用的文本显示控件，常用的就是 Label 文本控件了，当然 PushButon 按钮类也是可以显示文本的。但是我们不需要按钮的功能，只需要显示文本。所以用 Label 文本控件就可以了。

如下图，在左边的控件图，找到 Label 控件，Label 控件在“Display Widgets”（翻译为显示小部件）下，选中 Label 控件，将其拖至②处中间的设计窗体里。然后在③处选中 label 对象，再在④处 label 对象的属性栏找到 text 文件属性，修改里面的文本为“Hello World!”。这样我们就已经显示“Hello World”在窗口里了。当然双击中间的 Label 控件也是可以修改其文本属性的，对初学者来说还是按步骤修改，这样学习“Ui 设计器”是如何使用的。这里我们已经完成显示“Hello World!”的设计了。



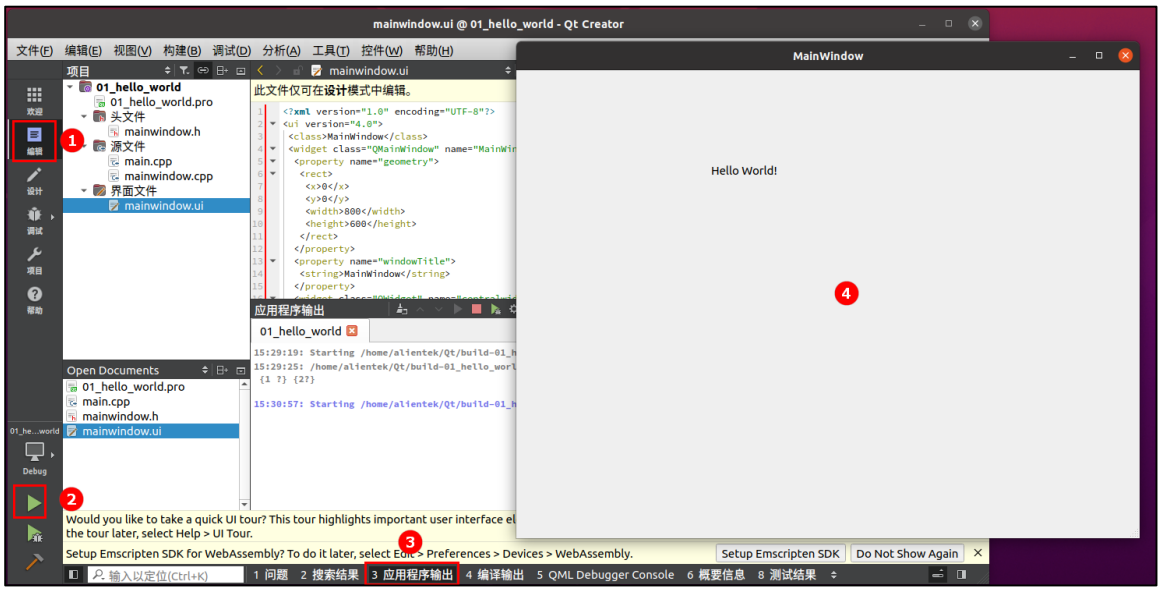
1.4.4 项目编译&调试&运行

完成上面的“Hello World!”后，我们肯定下一步是想编译运行我们的程序了，看看效果如何！

在 Ubuntu20.04，若 Ubuntu20.04 上有很多编译器，Qt Creator 可能默认没有指定哪种编译器，此时我们需要手动指定使用哪种编译器。点击“工具”》“外部”》“配置”，弹出的窗口，我们手动指定编译器。如不指定，Qt Creator 不知道用哪种编译器去编译，这里我们选择 clang 轻量级编译器或者 Ubuntu 自带的 GCC x86 64 bits /usr/bin 都可以，目的就是选择一个编译器而已。笔者此时没有选择，因为这个版本 Qt Creator 已经默认自动检测编译器并选择上了。注意，如果你看到 C/C++编译器这里为空，请回到文档最前面，输入本文档需要安装的软件/库指令。



按如下步骤，先返回编辑页面，再点击②处左下角的绿色三角符号（或者按 Ctrl + R 快捷键编译且运行），编辑时会输出编译信息，点击③处的编译输出窗口里，可以看到编译时的编译信息，能看到编译警告、编译错误、编译链接、相关编译器编译过程等。



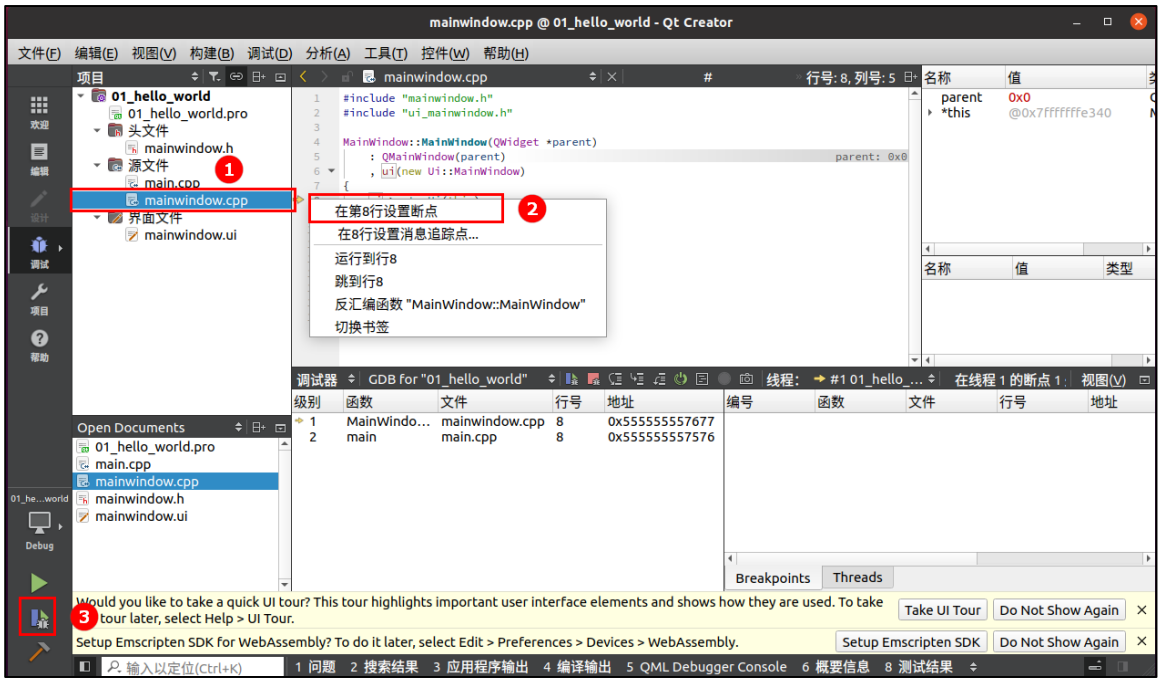
在 Ubuntu 下刚装的 Qt Creator，毫无意外，第一次编译时会报“Cannot find -lgL”的错误。由于 Qt5.0 的库默认会链接到 OpenGL，但是在 Ubuntu 机器上没有安装 OpenGL，所以们需要在 Ubuntu 下安装 OpenGL Library。在 Ubuntu 终端下输入如下指令。如果你在文档前面已经安装过 libglu1-mesa-dev 则无需安装。其中 libglu1 大写为 LIBGLU1，最后一个为数字 1，不是“L”。有些用户在此犯错。

```
sudo apt-get install libglu1-mesa-dev
```

安装完成后，我们再点击左下角的绿色三角形试试，可以看到如下结果！在下图②处已经看到没有报错信息了。如果还有其他报错，建议回到第一章 C++环境设置里寻找 C++环境配置的相关指令安装好相应的库。可能初学者跳过了 C++就直接到 Qt 这部分了，Qt 也依赖 C++的环境的！

运行的结果如④处，弹出一个窗口，中间就是我们设计的 Hello World!

如果需要调试，请先关闭上面的“Hello World!”窗口，点击右上角的“x”关闭即可！再按如下步骤点击，在程序里右键设置断点，再点击左下角③处的 Debug 绿色三角符号按钮。不过我们很少使用调试，这种简单的程序完全是可以预知它的运行过程与结果的，如果我们想要知道它的运行过程，是可以使用这个调试的功能去了解 Qt 程序的运行流程的！一般我们都是用 qDebug()打印调试。



第二章 交叉编译环境搭建

为了能够在 Ubuntu 下 qt creator IDE 上开发、编译 ATK-DLRK3568 平台（ARM 64bit）Qt 应用程序，我们需要对其进行相应的配置，包括 qmake、ATK-DLRK3568 交叉编译器等配置。

2.1 安装交叉编译工具链

请参考【正点原子】基于 Buildroot 系统_交叉编译器安装与使用参考手册第一章进行安装 。

2.2 命令行交叉编译 Qt 项目

安装完成交叉编译工具，这里我们需要检索这个交叉编译工具是否能用！所以我们需要测试一个 Qt 项目看看能不能用。

根据 2.1 小节，每次在新的终端使用编译器前都需要执行下面的指令，使能环境变量。

以 ATK-DLRK3568 为例：

```
export PATH=$PATH:/opt/atk-dlrk356x-toolchain/bin
```

2.2.1 查看 qmake 与 Qt 版本

执行下面指令查看 qmake 版本与 Qt 版本。可以看到交叉编译工具链里 qmake 版本为 3.1，qt 版本为 5.15.2。

```
/opt/atk-dlrk356x-toolchain/usr/bin/qmake -v  //或者直接输入 qmake(不建议，可能环境变化找不到，输入 qmake 的绝对路径比较保险。)
```

```
alientek@ubuntu:~/Qt/untitled$ /opt/atk-dlrk356x-toolchain/usr/bin/qmake -v
QMake version 3.1
Using Qt version 5.15.2 in /opt/atk-dlrk356x-toolchain/aarch64-buildroot-linux-gnu/sysroot/usr/lib
alientek@ubuntu:~/Qt/untitled$
```

以当前路径下有一个 untitled.pro（未命名的 Qt 项目）为例。就是新建的一个简单 Qt5 项目。

```
alientek@ubuntu:~/Qt/untitled$ ls
main.cpp  untitled.pro  untitled.pro.user  widget.cpp  widget.h  widget.ui
alientek@ubuntu:~/Qt/untitled$
```

执行 qmake 生成编译 Qt 项目所需的 Makefile，然后我们就可以编译 Qt 项目了。

以 ATK-DLRK3568 为例：

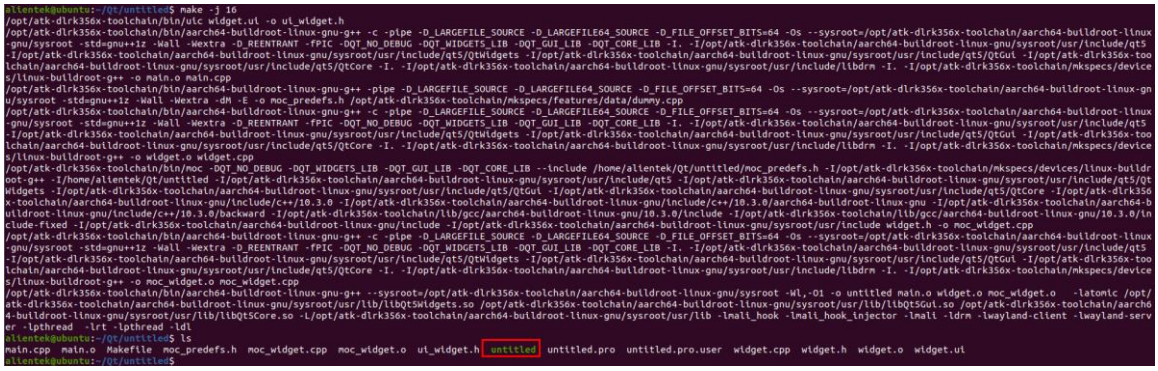
```
/opt/atk-dlrk356x-toolchain/usr/bin/qmake
```

```
alientek@ubuntu:~/Qt/untitled$ /opt/atk-dlrk356x-toolchain/usr/bin/qmake
Info: creating stash file /home/alientek/Qt/untitled/.qmake.stash
alientek@ubuntu:~/Qt/untitled$
```

2.2.2 编译 Qt 程序

直接输入 make 进行编译。

```
make -j 16  // -j x，x 代表你分配给你虚拟机的核心数。当然写大了也没关系，make 会根据实际情况去编译 Qt 项目。
```



如上，编译生成的可执行文件 untitled 就可以拷贝到正点原子 ATK-DLRK3568 出厂 Linux 系统上运行了。直接在出厂 linux 系统上./untitled 执行。

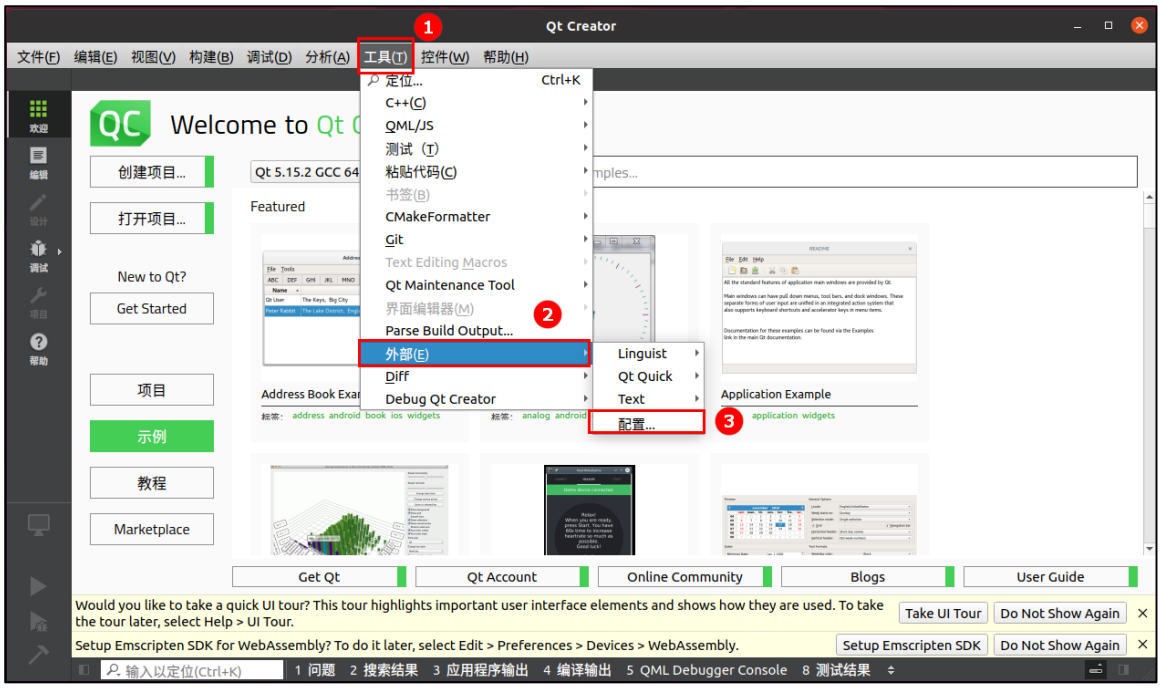
2.3 QtCreator 开发嵌入式平台 Qt 应用程序

很多开发者在编写 Qt 应用程序后，想在开发板上运行试试效果。毕竟我们在 Ubuntu 下的 Qt Creator 下开发的程序最终还是需要在开发板上运行的，如果你单纯只是开发 Ubuntu 上运行的 Qt 程序，那么你就没有必要搭建嵌入式平台的 Qt 应用程序了。

用命令行有个不好的地方，编写完 Qt 项目了，需要打开终端，再执行命令去编译，这样比较麻烦。有没有方法搭建一个开发套件（Kit）的方法呢，好吧，不废话了，我们就来使用 Qt Creator 搭建开发嵌入式平台 Qt 应用程序吧。

2.3.1 配置 ARM 交叉编译器

首先，我们在 Ubuntu 上打开 Qt Creator，点击顶部的菜单栏的“工具”》“外部”》“配置”。



可以看到⑥处，这个就是 Ubuntu 上桌面应用开发套件，也就是开发的 Qt 应用程序可以在 Ubuntu 上运行起来。通常我们都是在 Ubuntu 上开发界面。如果有些代码在 Ubuntu 上运行是没有效果的，也就是这些代码是与开发板硬件有关的，那么我们就必须编译这个程序到开发板上

运行观察运行结果。



以 ATK-DLRK3568 为例：

现在只有两个 kit，我们需要增加一个 kit，增加我们 ATK-DLRK3568 的配置。点击编译器选项，可以看到在 Ubuntu 上就有很多编译器，有 C 编译器还 C++编译器，这都是系统检测到的。然后点击添加按钮添加我们的 ATK-DLRK3568 平台的交叉编译器。在 2.1 小节我们已经安装了交叉编译器。所以我们在这一步就要用到这个交叉编译器了。



我们选择 GCC，为什么选择 GCC，因为我们用的编译器名字就是 GCC 类型的，然后我们首先配置 C++，也就是编译 C++的编译器。

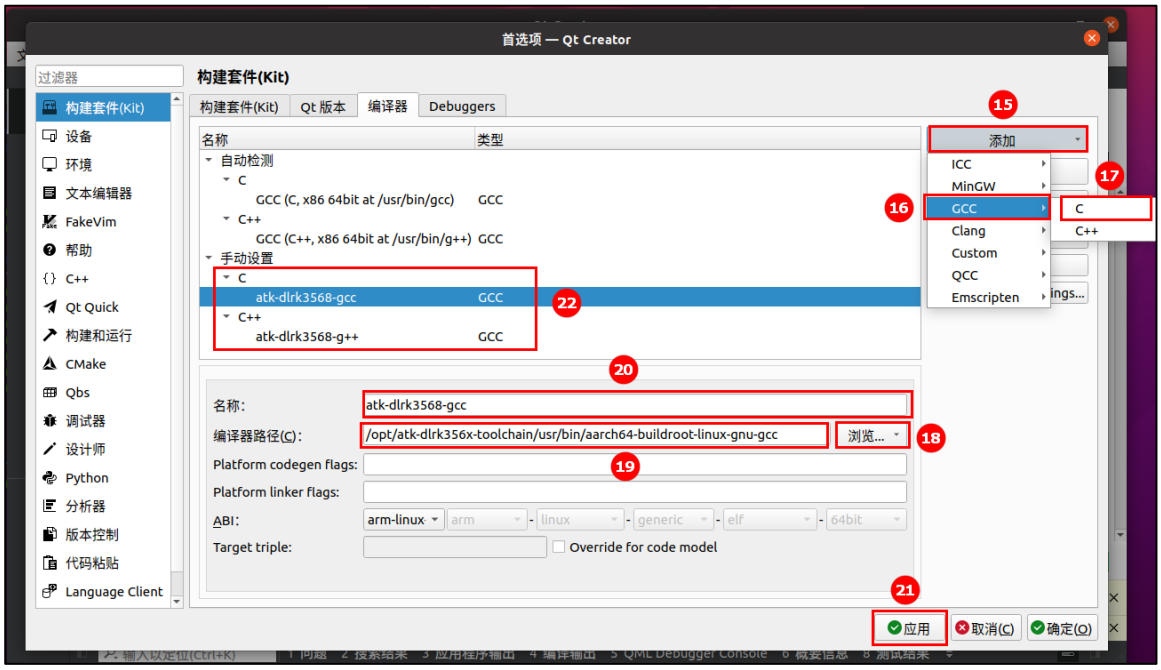


以 ATK-DLRK3568 为例（其他板子可参考，路径相似，下面的命名，请根据各自的板子，重新命名即可。32 位的 CPU 如 3506，g++为 arm-buildroot-linux-gnueabihf-g++而不是 aarch64-buildroot-linux-gnu-g++，找相似的即可。）：

点击后，我们需要将弹出的界面拉大一点。否则你就看不见⑪浏览路径的按钮了。在第⑫步中，这个路径是/opt/atk-dlrk356x-toolchain/usr/bin/aarch64-buildroot-linux-gnu-g++。如果你的交叉编译器也是按 2.1 小节安装在这个路径，那么我们的结果是一样的。如果你不想安装交叉编译器，那么你也是可以使用 ATK-DLRK3568 SDK 里的交叉编译器的，前提你要知道交叉编译器的路径在哪里！这里我们就不多说了。建议使用我们提供的交叉编译器，因为你可能编译 sdk 失败等原因导致编译不能使用的情况也是时有发生！在第⑬步我们需要自定义这个编译器的名称，我们的 ATK-DLRK3568 开发板就叫它 atk-dlrk3568-g++吧！写小写的，比较好认，当然你自定义它为其它名字也可以！

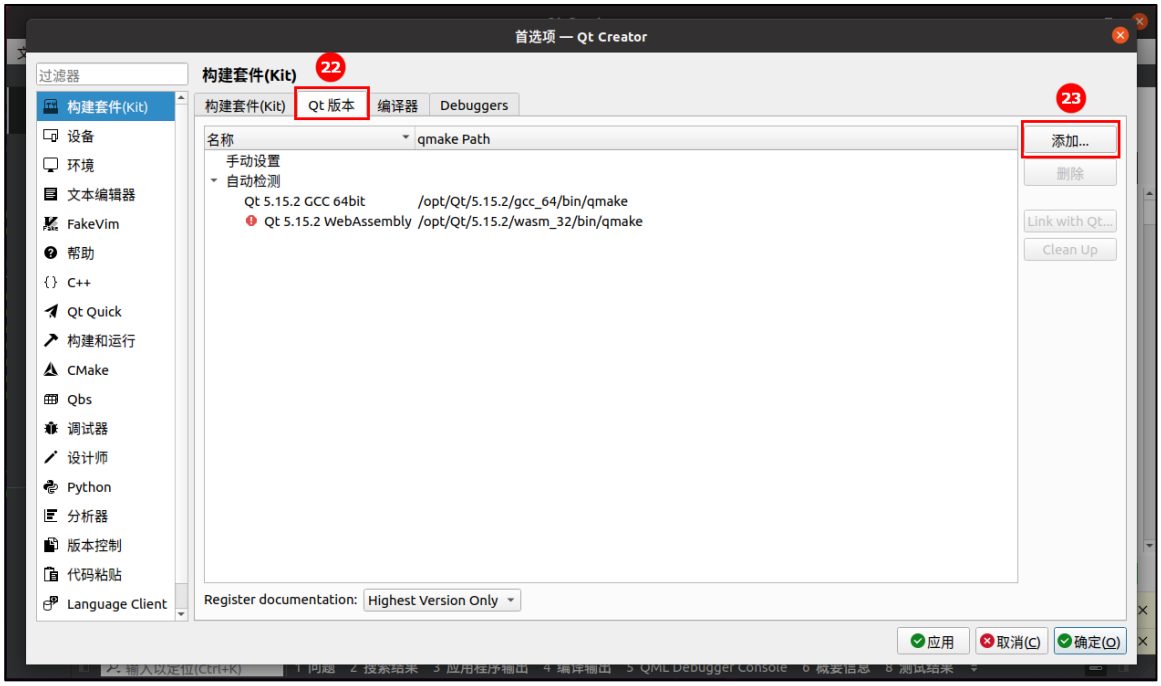


在上面配置完了 C++，我们一定要点 Apply（应用），现在我们来配置 C。重复上面的步骤。添加 C 编译器。下图同理，（32 位的 CPU 如 3506，g++为 arm-buildroot-linux-gnueabihf-g++而不是 aarch64-buildroot-linux-gnu-g++，找相似的即可）。gcc 这个路径是/opt/atk-dlrk356x-toolchain/usr/bin/aarch64-buildroot-linux-gnu-gcc。自定义的名称为 atk-dlrk3568-gcc。点击应用 Apply 即可。可以看到第 22 步中，我们已经添加了两种编译器。

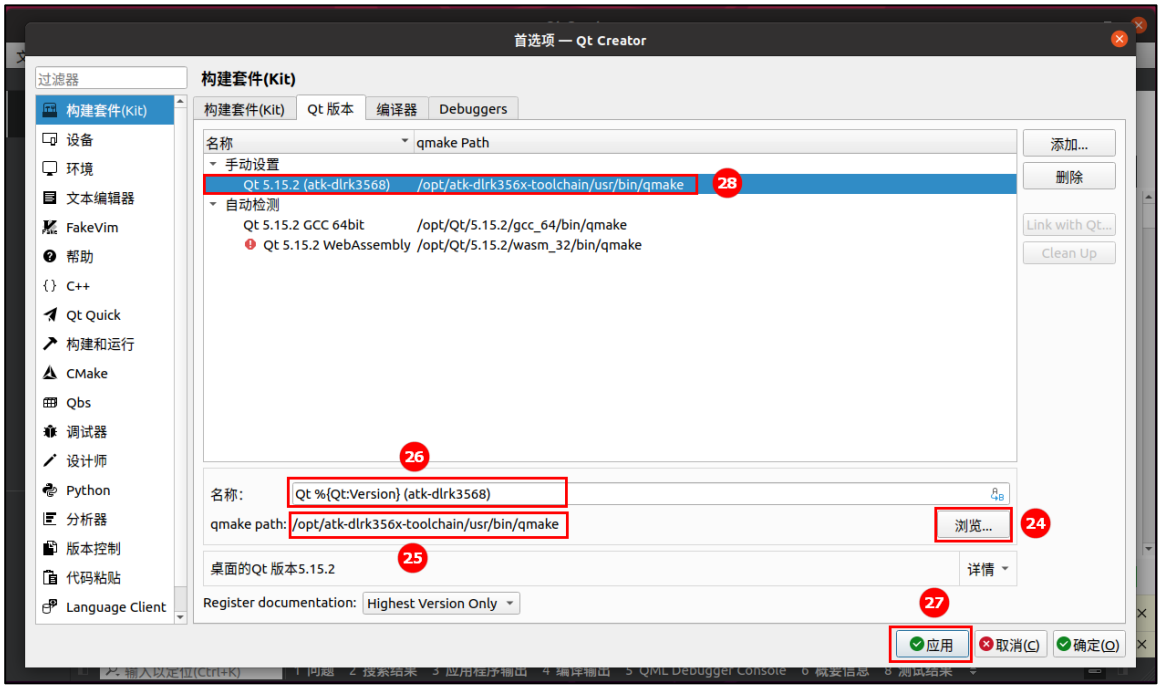


2.3.2 配置 qmake

上面配置完了交叉编译器，现在我们来配置 qmake，当然这个配置的顺序不是固定的，你可以先配置 qmake 再配置交叉编译器也没有关系。点击 Qt 版本，然后点击添加。



如下图，添加 qmake，如果你的交叉编译器也是和笔者的安装路径一样的话，qmake 的路径为/opt/atk-dlrk356x-toolchain/usr/bin/qmake。我们在第 26 步改写名称为 Qt %{Qt:Version} (atk-dlrk3568)。点击 Apply 应用！别忘记了！



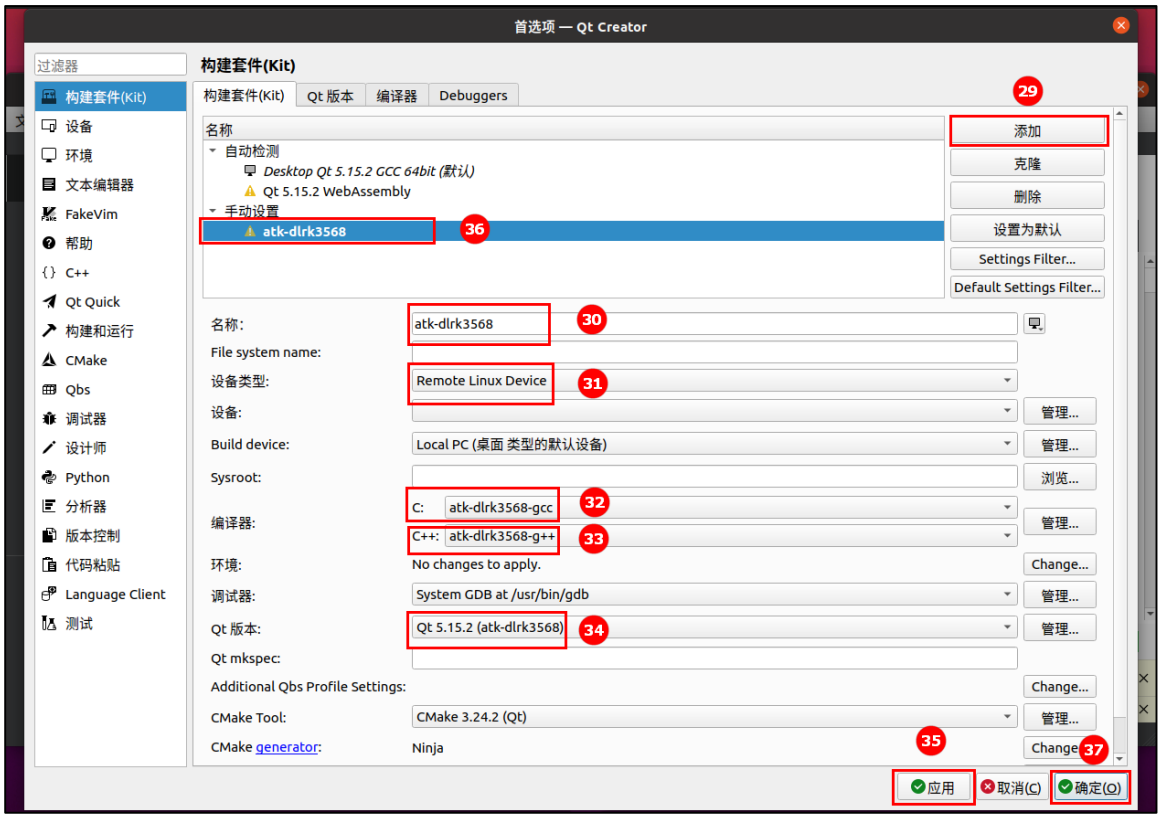
2.3.3 配置 Kit

这一步我们就需要添加我们自己的交叉编译套件了，点击添加。（注意：此界面需要用鼠标拉大，否则我们看不到完整界面。）

以 ATK-DLRK3568 为例：

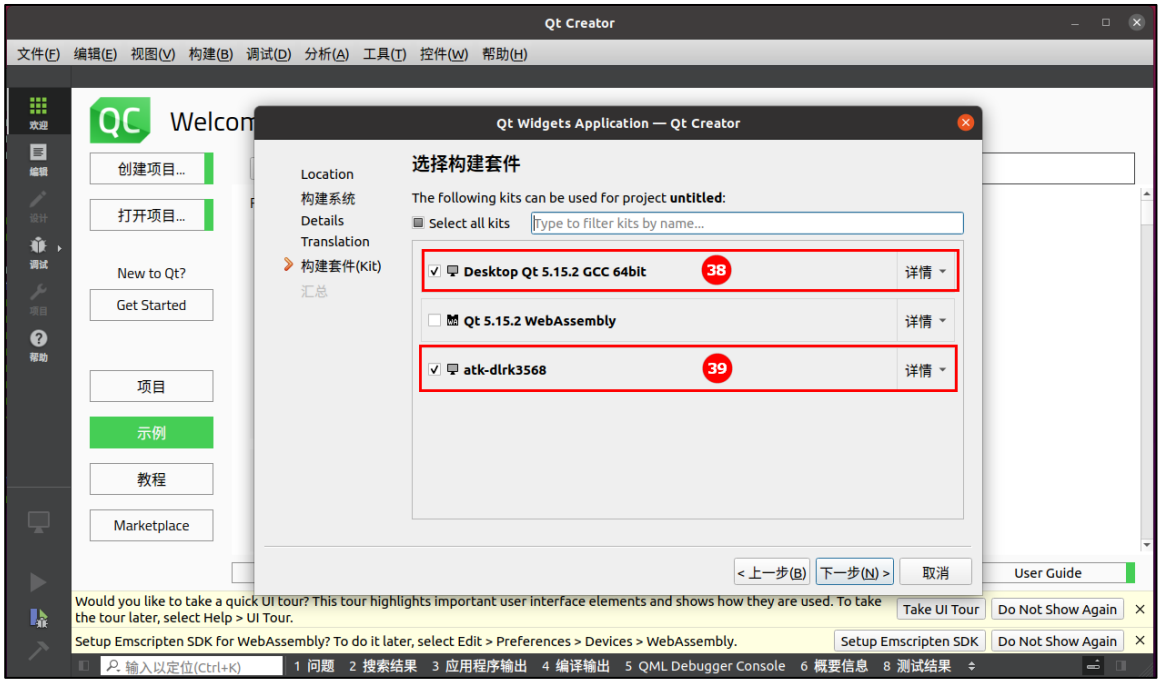
序号 30：这里我们命名为 atk-dlrk3568，这个就是我们开发板的名称了，写小写感觉比较顺眼。
序号 31：我们选择 Remote Linux Device（远程 Linux 设备）
序号 32 和 33：我们选择在 2.5.1 配置的交叉编译器类型
序号 34：我们选择在 2.5.2 配置的 qmake

可以看到下图中的 kit 只有一个黄色的叹号，（黄色的叹号只是我们的设备还没配置，所以出现黄色的叹号），说明我们已经配置成功了，没有错误。然后我们点击应用 Apply 再点击确认 OK 即可！

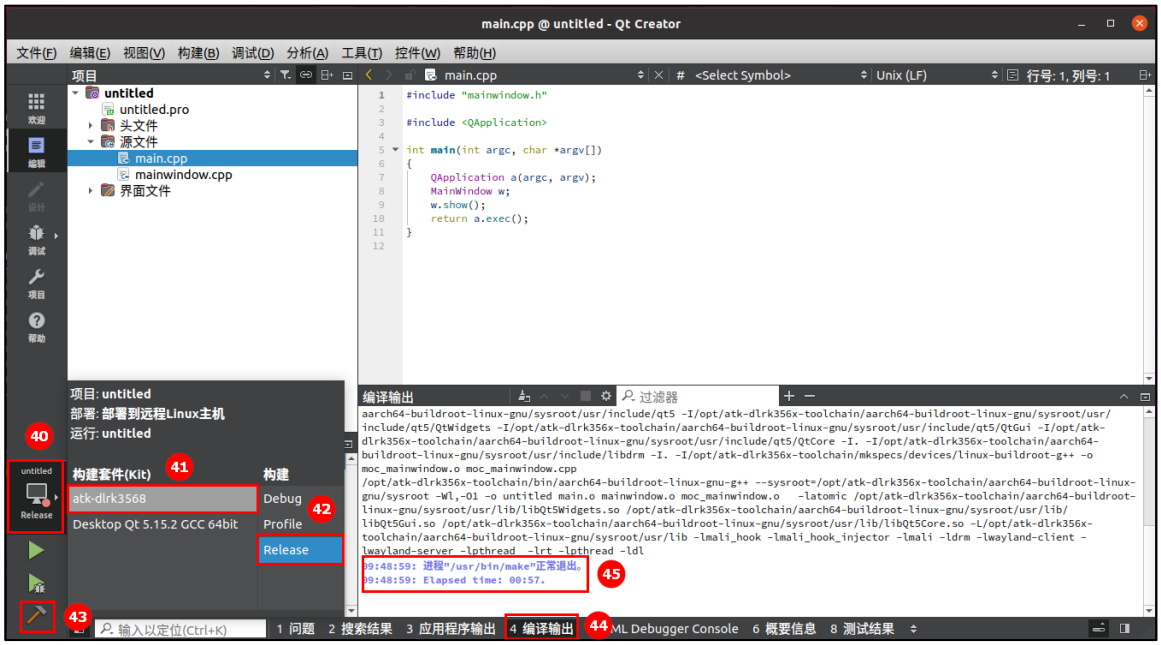


2.3.4 编译测试

现在我们新建一个简单的项目来测试。笔者快速创建了一个项目，我们在创建项目时，需要选择我们在上一步搭建好的编译套件。这里我们两个都选择，我们可以编译 Qt 程序在 Ubuntu 运行，又可以编译 ATK-DLRK3568 平台上运行的 Qt 应用程序了。



点击左下角的构建套件，我们选择 atk-dlrk3568 编译套件，选择 Release 或者 Debug 版本，注意我们点击 43 步的小锤子进行构建，不要点击运行（绿色三角形）。在编译输出窗口中可以看到 45 步中使用了我们在 2.1 小节中安装的交叉编译器，编译无错误！



编译生成的可执行文件在 build-项目名称-atk_dlrk3568-Release/下。请自行查看。拷贝到开发板系统上执行即可！

2.4 远程部署 Qt 应用程序

以 ATK-DLRK3568 为例：

第 2.3 小节中，使用“atk-dlrk3568”构建、编译的应用程序无法在 Ubuntu 本机上运行，可以运行在 ATK-DLRK3568 平台，但是需要我们将可执行文件拷贝到 ATK-DLRK3568 开发板，

然后再去执行它；这其实对于开发者来说非常不方便，因为它非常浪费时间，每次想要看下运行效果都需要将可执行文件拷贝到开发板上，比较麻烦！

本章我们就向大家介绍如何通过 qt creator 将应用程序远程部署到 ATK-DLRK3568 开发板上，也就是不通过我们人工拷贝的方式，我们直接在 Ubuntu 主机端 qt creator 软件上点击运行（或者快捷键）就可以实现 Qt 程序自动“跑到”开发板上运行，直接在开发板上查看程序运行效果！

2.4.1 配置运行设备

在 2.3.3 小节，还记得我们配置 kit 时有个黄色的感叹号吗？因为我们没有配置运行设备，你编译的 Qt 程序没有设备可以运行，所以就报黄了。默认的 Desktop Qt 5.15.2 GCC 64bit 这个编译套件配置了运行设备，就是 Ubuntu 桌面，它编译的程序就能跑在 Ubuntu 桌面上。我们配置的 atk-dlrk3568 编译套件需要配置运行设备，这个运行设备就是我们的开发板。

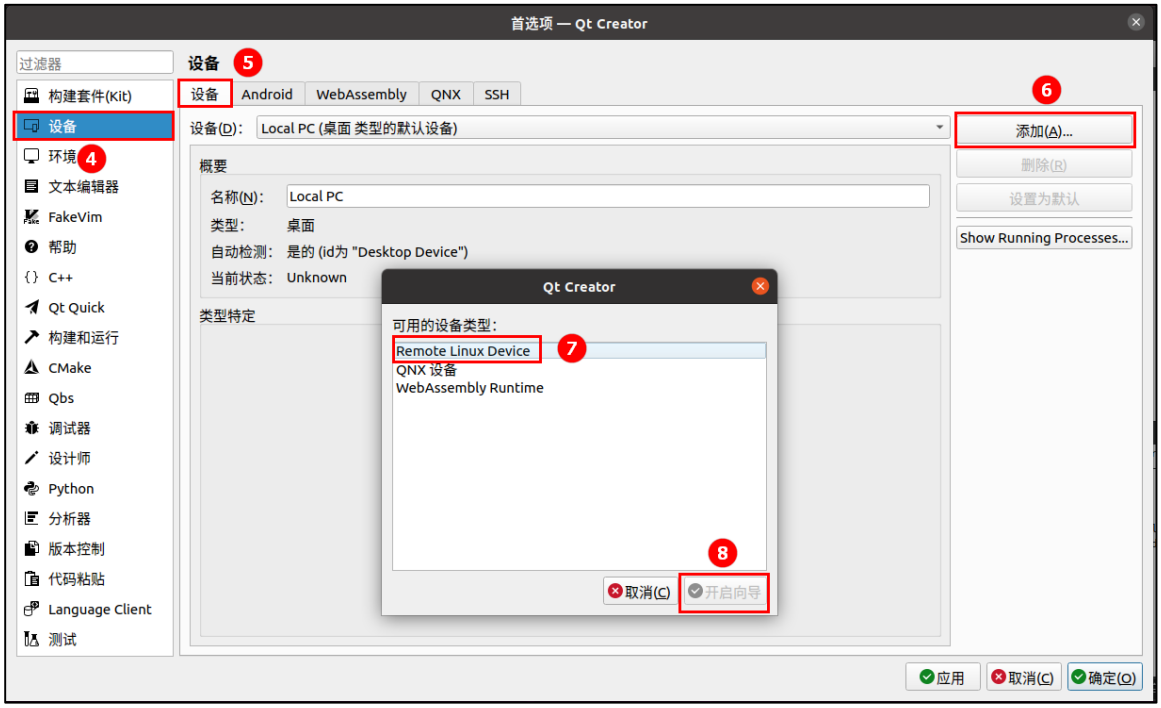
好，现在我们来创建运行设备。这个运行设备不在我们的 Ubuntu 上面，那么我们通过什么方式来连接我们 ATK-DLRK3568 开发板设备呢，答案是网络！

同理我们点击工具》外部》配置选项条目，如下。



在创建设备之前，保证 ATK-DLRK3568 开发板当前正运行 linux buildroot 系统，并且开发板已经通过网线连接到了路由器设备。

我们开始创建一个设备，点击设备条目，点击添加。按如下步骤添加一个通用的 Linux 设备。

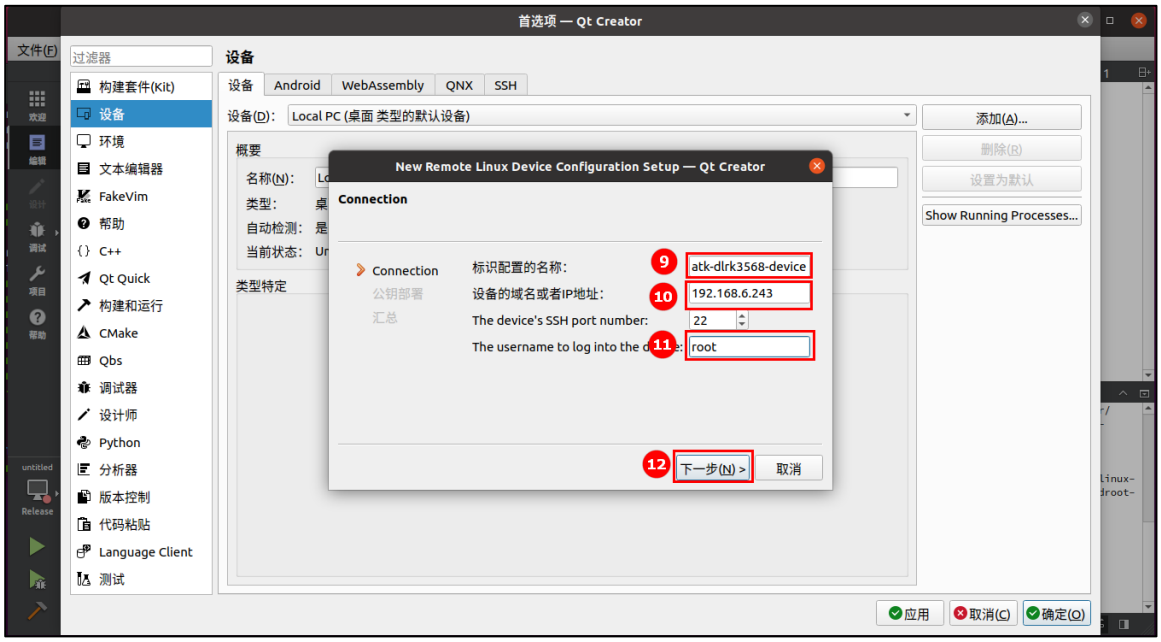


第⑨步将这个设备命名为 atk-dlrk3568-device，名字随你喜欢。

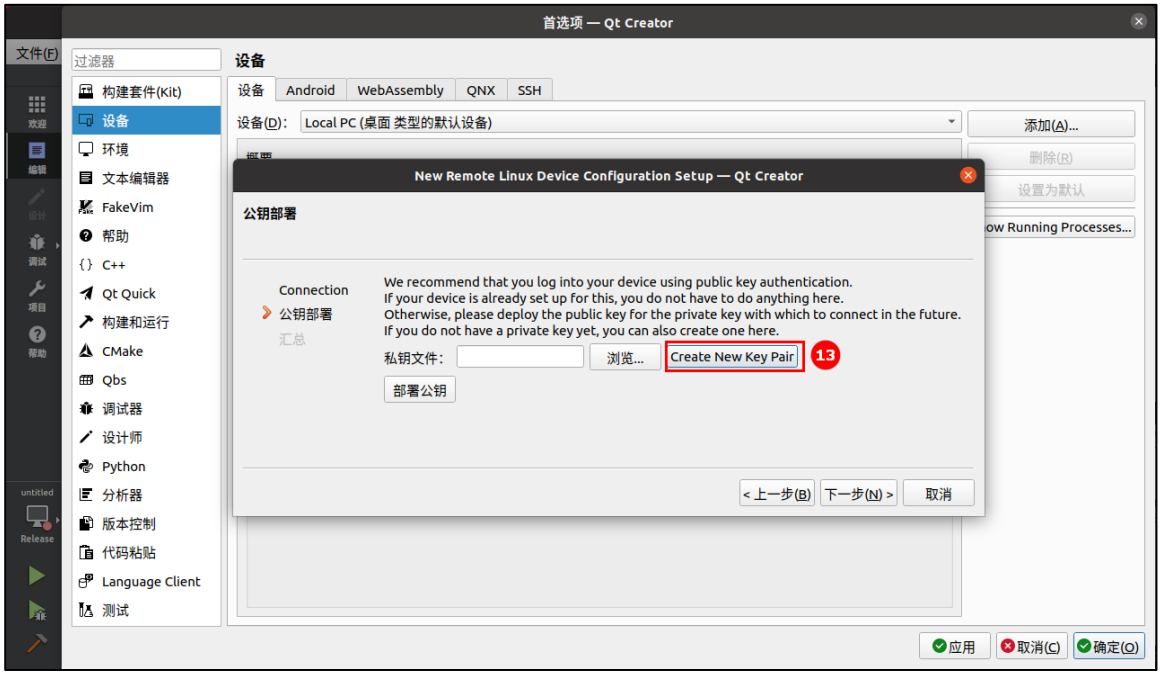
第⑩步填上开发板的 ip 地址，填写你自己开发板的 ip 地址。

第⑪步设备登录开发板的用户，我们设备为 root 用户登录。默认开发板就有 root 用户。

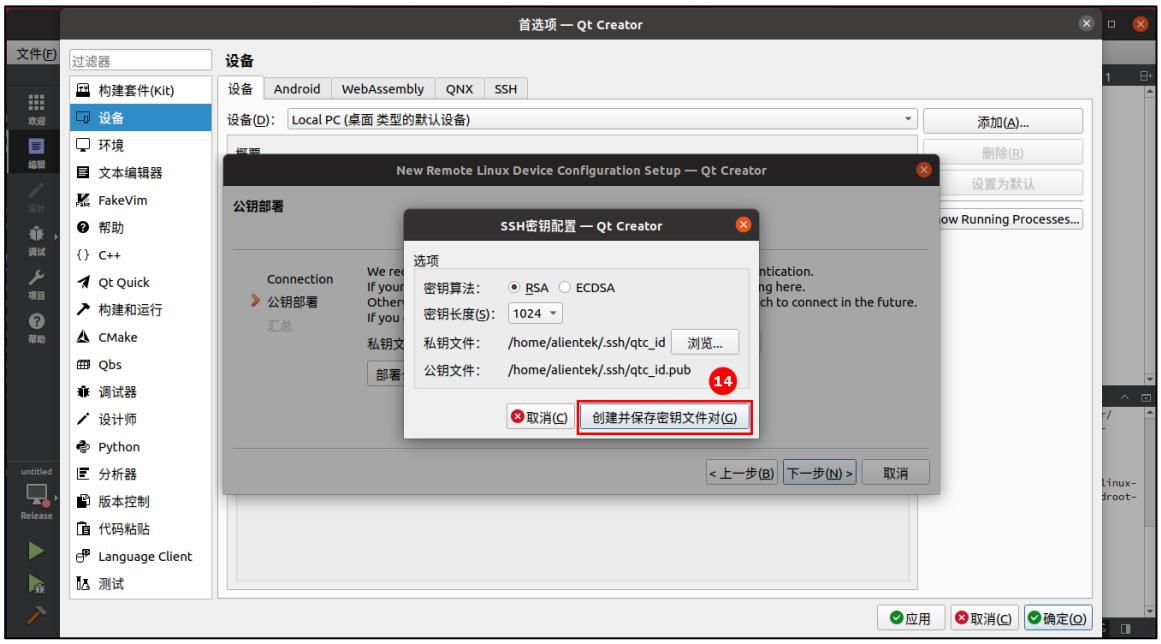
点击下一步。



点击创建密钥文件。



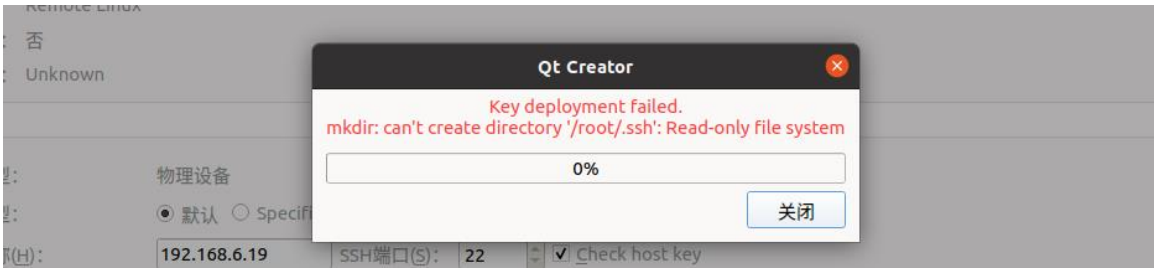
默认选项即可，然后点击创建密钥文件对。



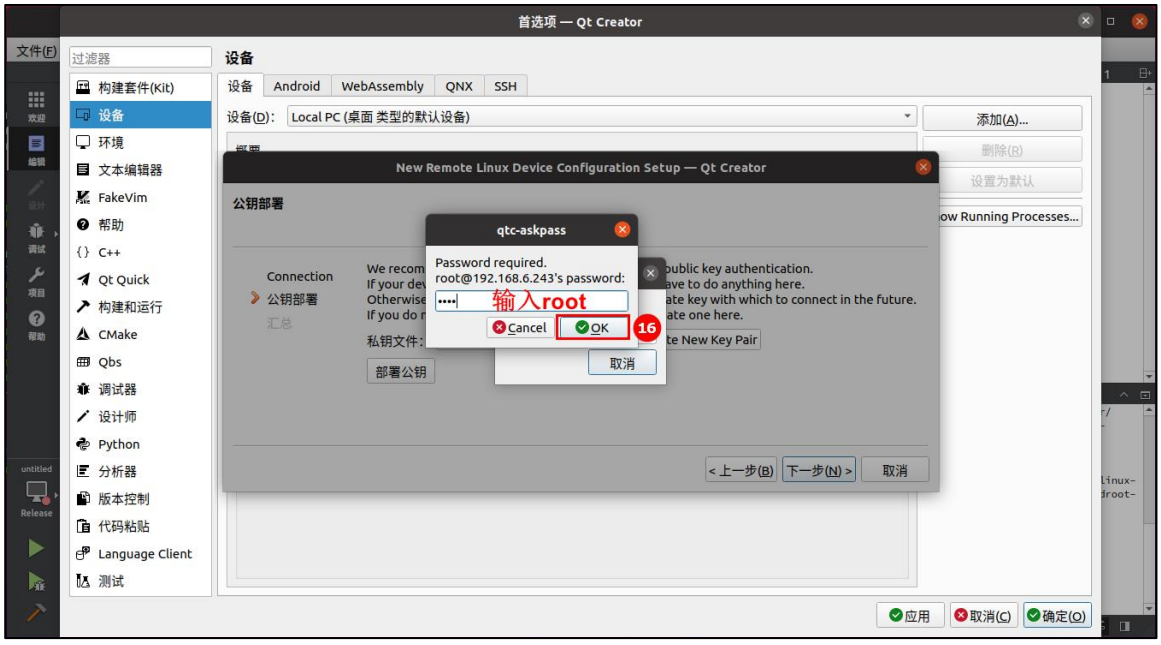
点击部署公钥。在上面一步，我们创建了密钥对，这个密钥对就好比一把锁和钥匙。我们可以将公钥代表钥匙，将钥匙部署到开发板上，于是锁和钥匙就“配对”了，这样他们就有权限通信了。



请注意，rk3506 nand 类型核心板出厂系统默认为只读文件系统，请先烧写可读写固件的系统再进行操作。否则你会看到如下报错。



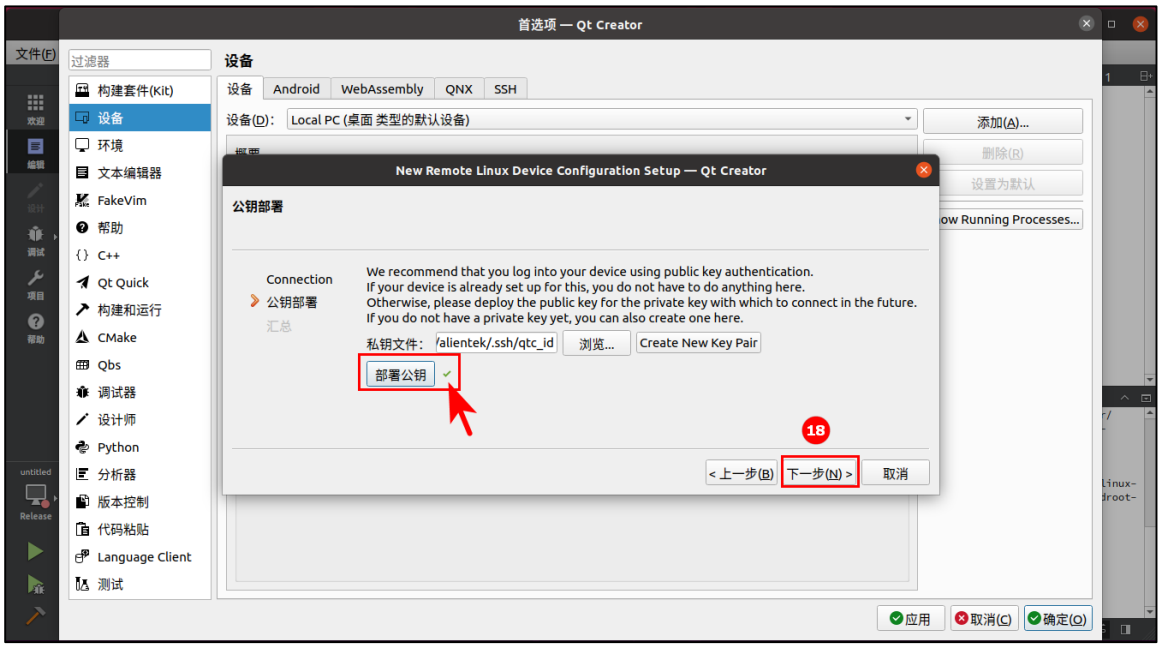
输入开发板 ssh 登录密码“root”,点击 Ok，即开始部署公钥。部署过程中请确认你板子的地址。如果开发板没有响应再尝试一次部署。



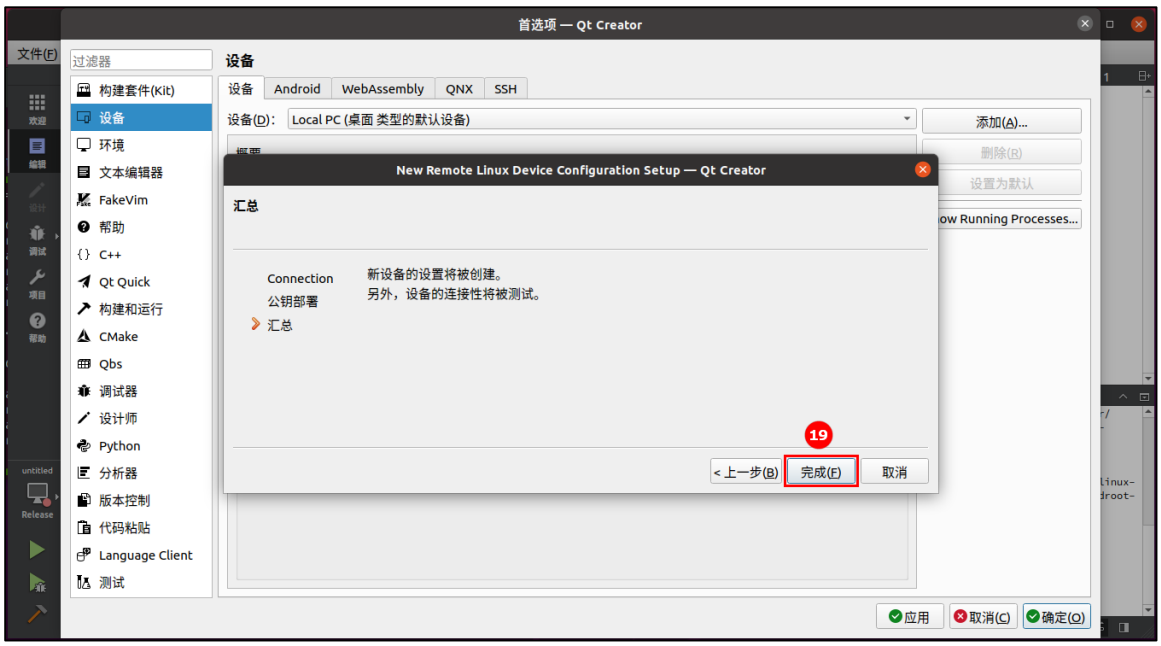
直到看到如下。



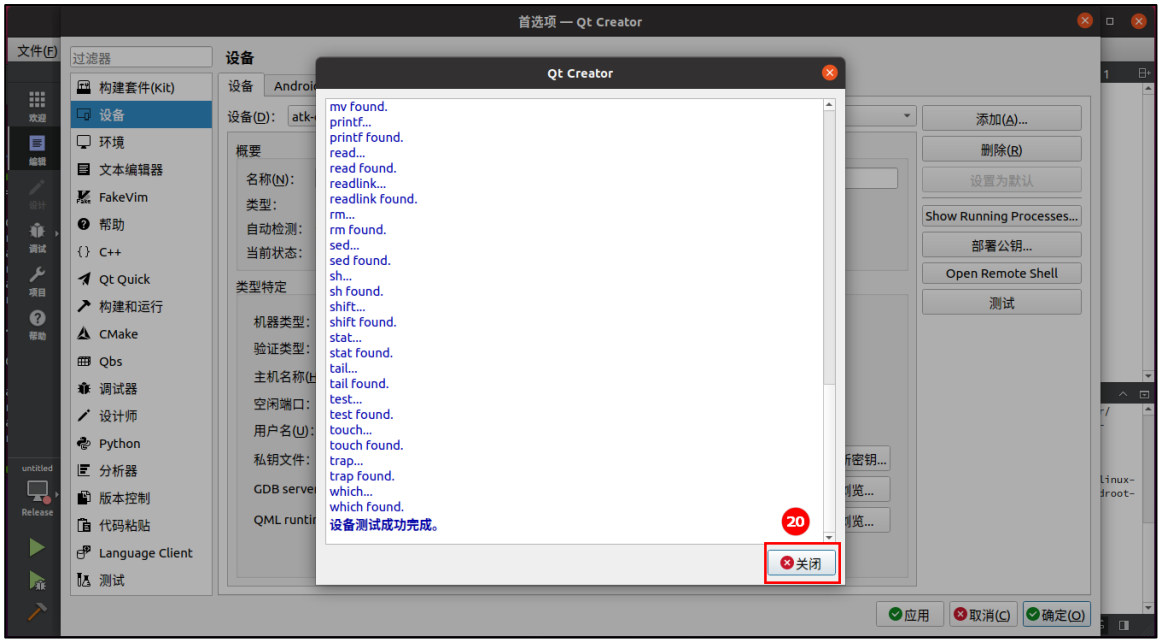
部署完公钥后，可以看到有个绿色的小勾。点击下一步。



点击完成后，设备连接性将被测试。



可以看到下图，设备测试成功。如有错误请检查开发板 ip 地址是否正确，看见（设备测试成功完成）Device test finished successfully 即可！若有其它报错可以忽略，点击关闭。



第 22 步可以看到你 atk-dlrk3568 前面的黄色小叹号不见了，23 步中，系统自动选择我们刚才配置的“atk-dlrk3568-device”点击确定，这样就完成了配置。

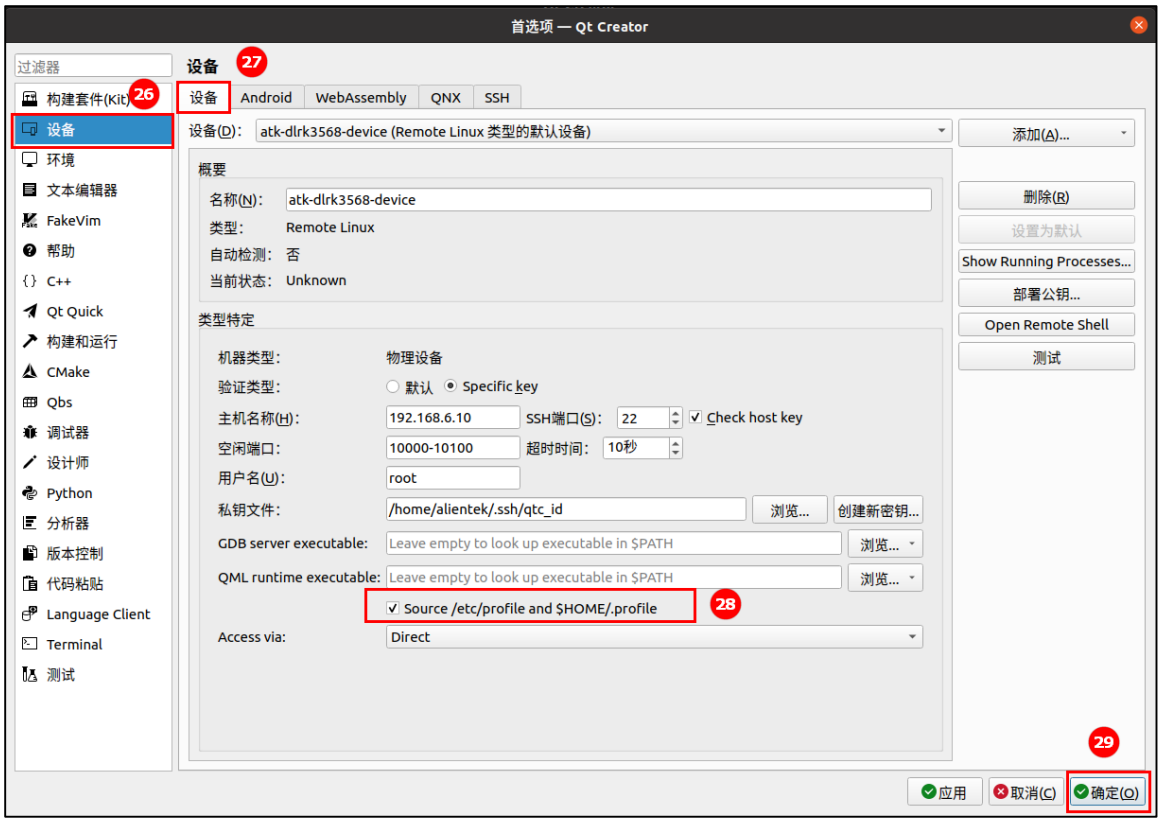


注意，笔者也提过，由于 Qt 官方的 QtCreator 进行了更新，笔者的 QtCreator 为 10.02，更新变为了 11.0.2，日后可能会更新。如果运行出现类似下面的错误。

```
22:31:03: Starting /opt/remove/bin/remove on atk-dlrk3568-device...
qt.qpa.plugin: Could not find the Qt platform plugin "eglfs" in ""
This application failed to start because no Qt platform plugin could be initialized. Rei
fix this problem.

Available platform plugins are: linuxfb, minimal, offscreen, vnc, wayland-egl, wayland.
```

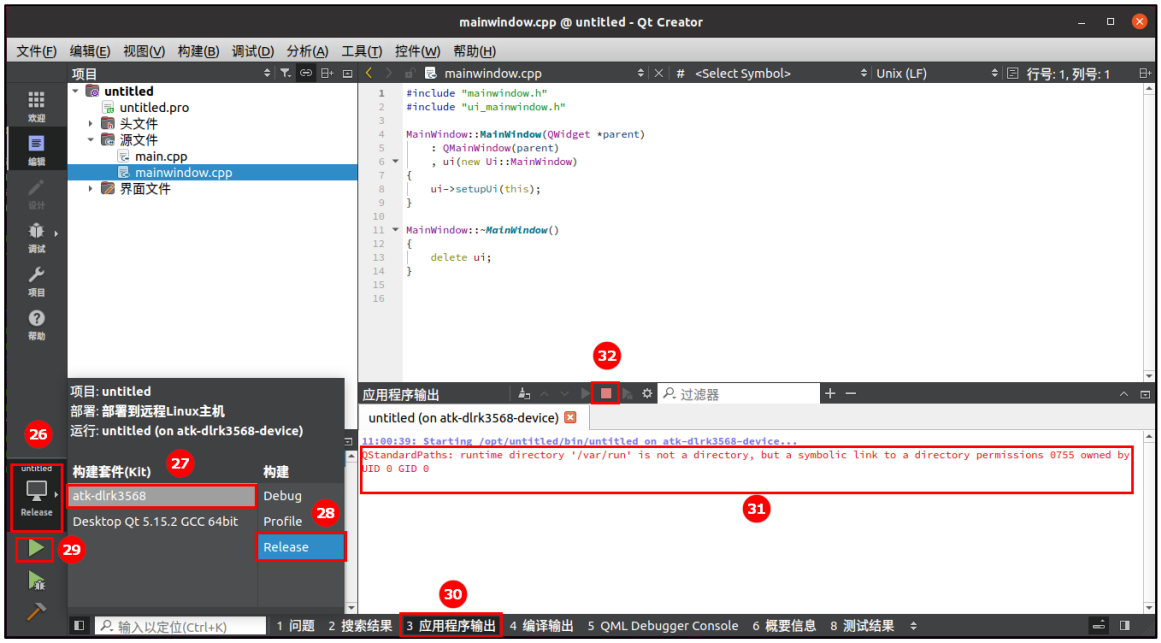
那么还需要按下图 28 步骤中的框框勾选（如果有这个可勾选的框框，没有就不用管），这里的意思是使用目标主机的环境变量。



2.4.2 运行设备测试

由于 ATK-DLRK3568 开发板出厂就跑了一个 Qt 主界面，请根据个人需要是否需要退出主界面后再运行设备测试。

如下图选择我们的 atk-dlrk3568 编译套件，选择 Debug 或者 Release，点击绿色的三角开，或者快捷键”Ctrl + r”即可在 ATK-DLRK3568 开发板上运行我们的 Qt 程序。由于我们这个例子为一个空白的窗口，所以你会在屏幕上看到一片白色，没有其它内容。运行成功在编译编译输出窗口看到如下图序号 30 处的信息。如遇到运行错误，请新建一个项目再按如下步骤测试。



若要停止这个程序，那么我们在应用程序输出窗口点击 32 步这个小红色矩形关掉这个程序即可。

至此嵌入式 Qt 开发环境搭建已经完成！

附录 A 常见问题

部署失败问题

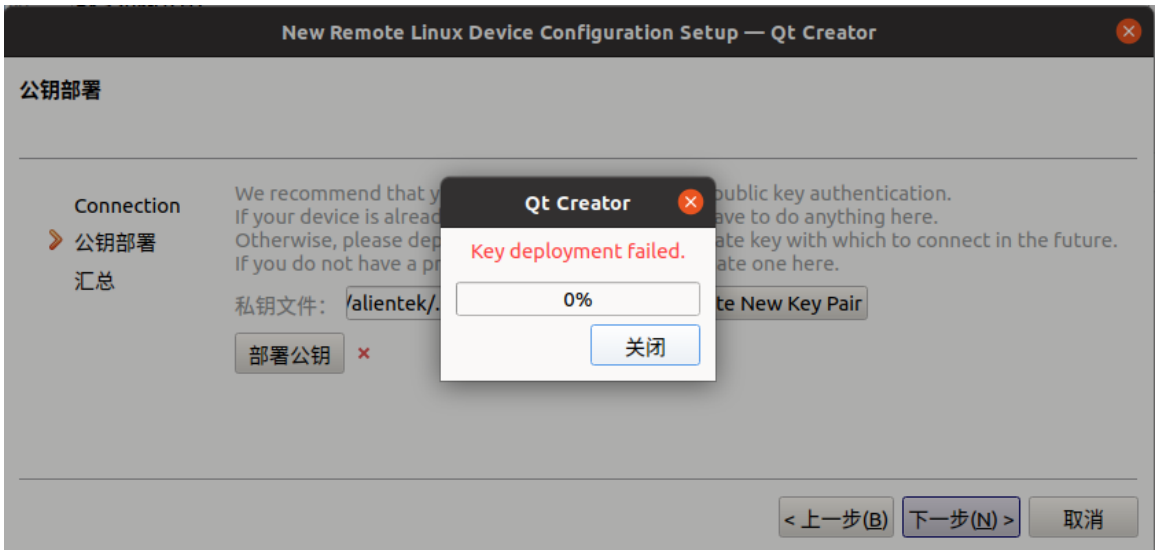
场景一:因为用户之前部署过公钥或者使用过 SCP 传输文件到开发板，然后用户又重新烧写了系统固件，新烧写的系统里，IP 还是原来的那个 IP。Ubuntu 的 ssh 的 known_hosts，known_hosts 文件通常包含了你已经连接过的 SSH 服务器的公钥。通常在你确定该 IP 地址的 SSH 服务器公钥已经改变（例如，服务器被重新安装了）或者你可能连接到了一个冒充的服务器时使用就会出现这个情况。

场景二:注意部署失败也可能是你的 IP 改变了！请确认部署的 IP 是否为开发板的 IP。

场景三:IP 冲突，在某些情况下在同一局域网内因为 IP 不够用，可能发生 IP 冲突。那么我们需要检查我们的开发板在拔掉网线的情况下，是否在 Ubuntu 用 ping 指令去 ping 这个主机地址，如果还有回复，那就是这 ip 地址已经被其他主机使用了，IP 已经冲突了！

场景四:如果你想快速解决，可试试换个网口，因为另一个网口你可能没用过就不会产生部署不成功的问题。

场景五：部署超时，所以以上四种场景没解决的话，我们就要考虑是不是部署超时失败导致的了。有些用户网络环境问题可能有较差导致部署公钥超时。默认超时为 10s。如果 10s 内板子没有响应则也会部署失败。那么我们将部署超时增加 30s 或者更大试试，详细解决看文档后面的解决方法。



同时我们使用 SCP 也会出现这个问题。

```
alientek@alientek:~$ scp h264.mp4 root@192.168.6.213:/
@@@
```

那么根据上面的提示，我们需要移除连接过的 SSH 服务器，执行下面指令。

ssh-keygen -f "/home/alientek/.ssh/known_hosts" -R "192.168.6.213" #红色部分请填写自己的 IP

```
alientek@alientek:~$ ssh-keygen -f "/home/alientek/.ssh/known_hosts" -R "192.168.6.213"
# Host 192.168.6.213 found: line 14
/home/alientek/.ssh/known_hosts updated.
Original contents retained as /home/alientek/.ssh/known_hosts.old
alientek@alientek:~$
```

然后我们可以愉快去部署公钥了。

超时失败问题解决：

